

Floating-Point Representation of Real Numbers:

From IEEE 754 to Modern Low-Precision Formats for AI Hardware — A Survey

Mahesha S

Compiler Engineer

mahesha.shivamallappa@gmail.com

Abstract

Floating-point arithmetic is the language in which computers approximate the continuum of real numbers, and the last decade of AI hardware design has turned that language into a moving target: 32-bit single-precision `float32` (formally `binary32` in the IEEE 754 standard) has been joined by 16-bit half-precision `float16` (formally `binary16` in the IEEE 754 standard), 16-bit `bfloat16`, 19-bit TensorFloat-32, a family of 8-bit floats, and now 6-bit and 4-bit “sub-byte floats” that are only usable at all because of blockwise *microscaling*. Each of these formats trades range, precision, and bit width against one another to achieve improved memory bandwidth and throughput on modern deep learning accelerators. This paper is a self-contained survey that walks that entire arc: it starts from first principles — why finite floating-point formats exist at all with a toy format being built up field by field — then proceeds through the IEEE 754 standard and its single-precision format in full detail, and then surveys every low-precision format in current production use: half precision, `bfloat16`, TensorFloat-32, the FP8 pair (E4M3/E5M2), the FP6 pair (E2M3/E3M2), and FP4 (E2M1). It then covers the Open Compute Project’s Microscaling (MX) format family in mathematical detail — scale computation, quantization, and dequantization — before turning to NVIDIA’s NVFP4 format, its two-level scaling schemes — tensor-scaling and block-scaling — and how it differs from strict MXFP4 quantization. The survey then follows these formats down through the rest of the stack: how they are realized in silicon on NVIDIA’s Blackwell architecture, how AI frameworks such as PyTorch and JAX represent them, and how a compiler toolchain — a graph compiler, MLIR, and a general-purpose LLVM compiler — actually lowers block-scaled low-precision computation onto that hardware, together with the ongoing IEEE P3109 effort to standardize 8-bit floating point for machine learning. This paper makes no novel technical contribution; its purpose is purely expository — to gather, in one consistently notated document, material that is otherwise scattered across IEEE standards text, vendor blog posts, whitepapers, and internal specification notes — so that a reader can start from nothing more than the real number line and walk away understanding exactly how an NVFP4 tensor gets encoded, scaled, and multiplied on a Blackwell GPU.

Contents

1	Introduction	4
1.1	Floating-Point Numbers and the IEEE 754 Standard	4
1.2	The Shift Toward Low-Precision AI Hardware	4
1.3	Motivation and Scope of This Survey	4
2	Preliminaries: Real Numbers and Floating-Point Approximation	5
2.1	The Problem: Infinitely Many Reals, Finitely Many Bits	5
2.2	A Hypothetical, Infinite-Precision Floating-Point Format	6
2.3	A Finite Floating-Point Format: The Tuple $(\beta, P, E_{\min}, E_{\max})$	6
2.4	Reserved Exponent Codes: Encoding Zero, Sub-normals, Infinity, and NaN	8
2.5	Terminology: Precision, Machine Epsilon, and ULP	9
3	The IEEE 754 Standard	10
3.1	Historical Evolution	10
3.2	General IEEE Binary Encoding	11
3.3	IEEE Single Precision (float32) in Detail	11
3.4	Rounding	13
4	Low-Precision Floating-Point Formats	15
5	16-bit and 19-bit Formats: The First Step Down	16
5.1	IEEE Half Precision (float16)	16
5.2	Google’s bfloat16	16
5.3	NVIDIA’s TensorFloat-32 (TF32)	17
5.4	Comparison	17
6	8-bit Floating-Point Formats (FP8)	17
6.1	The Two FP8 Encodings: E5M2 and E4M3	18
6.2	How FP8 Is Used in Deep Learning	19
7	Sub-Byte Floating-Point Formats: FP6 and FP4	19
7.1	Shared Encoding Rules	20
7.2	FP6: E2M3 and E3M2	20
7.3	FP4: E2M1	20
8	The Idea of Scaling	22
9	Block Scaling: The Microscaling (MX) Format Family	23
9.1	Anatomy of an MX-Compliant Block	23
9.2	The MXFP Scale Type: E8M0	24
9.3	The MXFP Scaling Block Size: $k = 32$	24
9.4	Block Scaling Factor	25
9.5	Efficient, Bit-Manipulation-Based Computation of the Scale Factor	27
9.6	Special-Case Conversion: Saturation vs. Overflow	31
9.7	Quantization	31
9.8	Dequantization	33
10	NVIDIA NVFP4: A Refined 4-bit Format	35
10.1	Two-Level Scaling: Tensor and Block	35
10.2	Why Two-Level Scaling?	35
10.3	Scaling Factor Computation	36

10.4 NVFP4 Quantization	37
11 Hardware Realization of Low-Precision Types	38
11.1 Hardware Challenges of Realizing Low-Precision Types	38
11.2 Mitigation: Packed Storage and Qualifier-Encoded Types	38
11.3 NVIDIA Blackwell: A Concrete Illustration	39
12 Framework-Level Realization of Low-Precision Types	40
12.1 Framework Challenges of Representing Low-Precision Types	40
12.2 Mitigation: Dtype-Level Packing and Object-Level Scale Metadata	41
13 Compiler-Level Realization of Low-Precision Types	42
13.1 Why the Two Ends of the Compiler Stack Stay Generic	42
13.2 Mitigation: MLIR as Low-Precision Lowering Engine	43
14 Standardization and Outlook	45
14.1 IEEE P3109: Standardizing 8-bit Floating Point	45
15 Conclusion	45

1 Introduction

1.1 Floating-Point Numbers and the IEEE 754 Standard

Every real number a computer works with, in practice, is a *floating-point* number: an approximation of some real value using a sign, an exponent, and a significand (also called a mantissa) — the same triplet that ordinary scientific notation already uses on paper. What paper-based scientific notation never has to deal with is a hard limit on how many bits represent that triplet: a computer must fix the number of bits for the exponent and the significand in advance, and that one constraint forces a set of further design questions — how to represent zero, how to signal an invalid or infinite result, what happens when a computation overflows the representable range, and how rounding should behave so that different machines agree with each other. The IEEE 754 standard, first ratified in 1985 and since revised twice, answered these questions for general-purpose computing and has been essentially unchallenged for three decades [4, 5, 7].

1.2 The Shift Toward Low-Precision AI Hardware

What *has* changed in the last decade is the workload. Deep neural network training and inference are dominated by dense linear algebra — matrix multiplications and convolutions — performed billions of times over, where the bottleneck is rarely the accuracy of any individual multiply-accumulate but almost always the amount of data that has to move through the memory hierarchy and the number of such operations a chip can issue per clock. This has created enormous commercial pressure to shrink the floating-point format itself: half the bits means half the memory traffic and, if the hardware is built for it, more arithmetic throughput per unit of silicon and power. That pressure produced, in quick succession, IEEE float16 (already part of the 754 family but rarely used for scalar computing), Google’s `bfloat16` [45], NVIDIA’s TensorFloat-32 [23], an entire family of 8-bit floating-point formats [17], and — more recently and much more aggressively — 6-bit and 4-bit floating-point formats that are only useful in practice because of an additional idea called *microscaling*: sharing one scale factor across a small block of values so that the per-value format only needs to represent a shape, not an absolute magnitude [31, 42]. NVIDIA’s Blackwell generation of GPUs pushes this further still with a bespoke 4-bit format, NVFP4, that refines the block-scaling idea specifically to recover accuracy that plain 4-bit microscaling loses [24].

1.3 Motivation and Scope of This Survey

The material above is not hard to find individually: the IEEE 754 text is public, vendor blogs describe `bfloat16`, TF32, and NVFP4 in varying levels of detail, and the OCP Microscaling specification is a freely downloadable PDF. What is harder to find is all of it *in one place, in one notation, building on itself*: a reader who understands IEEE float32 field-by-field should be able to read straight through to understanding exactly how a 4-bit NVFP4 tensor is quantized, scaled, and dequantized on a Blackwell tensor core, without having to reconcile five different notations for “exponent bias” along the way. That is the gap this survey tries to close.

Concretely, this paper:

- builds up the idea of a finite floating-point format completely from first principles, starting from the real number line and a hypothetical infinite-precision representation, before ever mentioning IEEE 754 (Section 2);
- covers the IEEE 754 standard’s history and its single-precision format in full bit-level detail, as the reference point against which every subsequent low-precision format will be compared (Section 3);

- surveys every “narrow” floating-point format in current production use for AI workloads: half precision, `bfloat16`, TensorFloat-32 (Section 5), the FP8 pair E4M3/E5M2 (Section 6), and the FP6 and FP4 sub-byte formats (Section 7);
- introduces *scaling*, the general idea that makes low-precision formats usable on real tensors in the first place (Section 8), then works through the mathematics of two concrete scaling schemes built on that idea: the OCP MX format family’s block microscaling, with its scale computation, quantization, and dequantization (Section 9), and NVIDIA’s NVFP4 refinement of it, including a worked comparison of why NVFP4 is more accurate than plain MXFP4 at 4 bits (Section 10);
- connects all of the above to their hardware realization (Section 11), then moves from hardware back up to software, covering how AI frameworks such as PyTorch and JAX expose and represent these low-precision types within their own tensor systems (Section 12), and how the compiler stack underneath those frameworks — a graph compiler, MLIR, and a general-purpose LLVM compiler — actually lowers that framework-level computation onto the hardware (Section 13);
- and covers the still-ongoing IEEE P3109 standardization effort for 8-bit floating point (Section 14).

A note on novelty

This is a survey and tutorial paper. It does not claim, and is not intended to contain, any new technique, format, or result. Every technical claim about an existing standard or product is drawn from and cited to a primary or vendor source; where this paper adds anything of its own, it is only exposition, notation, worked examples, and the figures used to connect the pieces — written for a reader who wants the whole story of floating-point, from the real number line to a Blackwell tensor core, told once, consistently, and in order.

2 Preliminaries: Real Numbers and Floating-Point Approximation

2.1 The Problem: Infinitely Many Reals, Finitely Many Bits

The real numbers $\mathbb{R}$ form a continuum: they are used to model continuous quantities such as distance, duration, or temperature, and between any two distinct reals $a < b$ there are infinitely many other reals, no matter how close a and b are. A computer, by contrast, has only a finite number of bits to represent any single value. Representing “all of $\mathbb{R}$ ” exactly is therefore impossible, and every computer number format is really a scheme for choosing a finite, well-spread subset $\mathbb{R}' \subset \mathbb{R}$ to represent $\mathbb{R}$ exactly, together with a rule for how to round every other real number in $\mathbb{R}$ onto the nearest member of $\mathbb{R}'$. A good such scheme should, informally:

- let $\mathbb{R}'$ cover as wide a span of magnitudes from $\mathbb{R}$ as the intended application needs (its *range*),
- space the points of $\mathbb{R}'$ closely enough together that rounding any value of $\mathbb{R}$ onto its nearest neighbor in $\mathbb{R}'$ introduces only a small error (its *precision*),
- be simple enough to implement efficiently in hardware, and
- be fast to compute with.

Floating-point arithmetic is, overwhelmingly, the scheme the industry settled on for general-

purpose computing, ahead of alternatives such as fixed-point arithmetic (a fixed number of fractional bits, good range only over a narrow, application-specific span) or logarithmic number systems (which trade multiplication for addition but make addition itself expensive).

2.2 A Hypothetical, Infinite-Precision Floating-Point Format

Before fixing any bit widths, it helps to write down floating-point representation the way scientific notation already does it. A real number x is represented as

$$x = \pm S \times \beta^E \quad (1)$$

where β is the *radix* or base of the number system (2 for binary, 10 for decimal, $\dots$), E is an integer *exponent*, and S is a real-valued *significand*, normalized so that $S \in [1, \beta)$. In base 10 this is exactly ordinary scientific notation:

$$365.25 = 3.6525 \times 10^2, \quad 0.00036525 = 3.6525 \times 10^{-4}, \quad -123456.789 = -1.23456789 \times 10^5.$$

Equation (1) places no bound on how many digits S may have, nor on the range of E — it is, in that sense, an *infinite-precision*, *infinite-range* floating-point representation, and that unbounded digit count is what lets it represent any *nonzero* real exactly, including irrational values such as π . Zero is the one value this normalized form cannot reach: since $S \in [1, \beta)$ excludes $S = 0$, no choice of (S, E) satisfies $x = 0$, so zero needs its own special encoding (Section 2.4). The entire remaining design problem of a real, finite-precision floating-point *format* is: how do we truncate S to a fixed number of digits and E to a fixed, bounded range, and what do we do at the edges of that truncation?

2.3 A Finite Floating-Point Format: The Tuple $(\beta, P, E_{\min}, E_{\max})$

A concrete, finite floating-point format is characterized by a tuple $(\beta, P, E_{\min}, E_{\max})$, where β is the radix (almost always 2 in hardware), P is the number of radix digits used for the significand (the *precision*), and $E_{\min} < 0 < E_{\max}$ bound the exponent. In binary ($\beta = 2$), a normalized significand always has the form $1.b_1b_2 \dots b_{P-1}$: since the only nonzero binary digit is 1, the leading digit is always exactly 1 and is never actually stored. It is an *implicit* or *hidden* bit, and only the $P - 1$ fractional bits $b_1 \dots b_{P-1}$ need to be represented.

A toy format.

To make this concrete, consider the toy format with $(\beta, P, E_{\min}, E_{\max}) = (2, 3, -1, 1)$: two fractional significand bits, and an exponent restricted to $\{-1, 0, 1\}$. Table 1 lists every normalized value this format can represent, positive and negative alike (zero and sub-normal values are excluded here, since their encoding needs the reserved-code trick introduced in Section 2.4).

Table 1: All 24 normalized values of the toy format $(\beta, P, E_{\min}, E_{\max}) = (2, 3, -1, 1)$, arranged by sign and exponent (rows) and significand (columns); every representable value has its own cell.

		Significand $S = 1.b_1b_2$			
		1.00 ₂	1.01 ₂	1.10 ₂	1.11 ₂
Sign +	Exponent E				
	$E = -1 (\times 2^{-1})$	0.5	0.625	0.75	0.875
	$E = 0 (\times 2^0)$	1.0	1.25	1.5	1.75
	$E = 1 (\times 2^1)$	2.0	2.5	3.0	3.5
Sign −	$E = -1 (\times 2^{-1})$	−0.5	−0.625	−0.75	−0.875
	$E = 0 (\times 2^0)$	−1.0	−1.25	−1.5	−1.75
	$E = 1 (\times 2^1)$	−2.0	−2.5	−3.0	−3.5

Real numbers that coincide exactly with an entry of such a table — like 0.5, 0.625, −0.5, −0.625, and so on above — are called *floating-point numbers* of that format; every other real number, such as 0.6, has to be rounded to its nearest neighbor in the table. Figure 1 plots the twelve positive values of this toy format on the real line; the twelve negative values are their mirror image about zero. Two things are immediately visible: first, the values are *not* evenly spaced — they cluster near 0 and spread out as magnitude grows, because each doubling of the exponent doubles the gap between consecutive representable values; second, within one exponent’s range the four values *are* evenly spaced, at a gap that depends only on E .

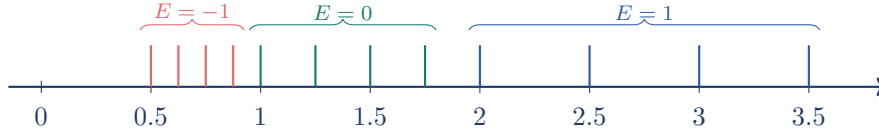


Figure 1: The twelve positive normalized values of the toy format, grouped by exponent. Gaps double with each increase in E ; within a fixed E , gaps are uniform.

Several immediate questions fall out of this toy example, and each has a direct analogue in IEEE 754:

- **Non-representable reals.** What happens to a value like 0.6, which falls strictly between two representable points? — This is a *rounding* question, governed by the format’s rounding mode (round-to-nearest-even being the IEEE default).
- **Zero.** None of the entries in Table 1 is 0, because a *normalized* significand is always ≥ 1 . Zero needs a special encoding.
- **Infinity.** Does the format need a dedicated encoding for a magnitude too large to represent at all — such as the true mathematical result of a division by zero — and if so, what bit pattern represents it?
- **Invalid results (NaN).** What should an operation with no sensible real-valued result, such as $0/0$ or $\sqrt{-1}$ in a real-only format, produce?
- **Sub-normal (denormal) numbers.** The toy format’s smallest positive normalized value is 0.5. But there is unused encoding space below it (significands with a leading 0 digit)

that can be used to represent smaller values, at reduced precision, rather than jumping straight from 0.5 to 0.

- **Overflow.** When an actual computation's result is too large for the format to hold, e.g., something > 3.5 in the toy format, what should actually happen: round it to the infinity encoding above, saturate at the largest finite representable value, or something else?
- **How wide should the exponent field be?** There is no single correct answer; it depends on the bit budget available and on the range of magnitudes the target application actually needs.

The next subsection answers four of these questions at once — zero, sub-normal numbers, infinity, and NaN — with a single idea: reserved exponent codes.

2.4 Reserved Exponent Codes: Encoding Zero, Sub-normals, Infinity, and NaN

So far E has stood for the true, signed exponent. In hardware, E is not stored directly; it is stored as an *encoded field* — a non-negative integer related to E by a fixed, format-specific *bias*:

$$\text{encoded field} = E + \text{bias}.$$

As the encoded field counts up from 0, the true exponent E it represents counts up too, one-for-one; biasing exists so that two exponents can be compared correctly with ordinary unsigned integer comparison, without separate sign handling.

IEEE 754 answers every question above except rounding with a single rule: two specific encoded exponent values are set aside for special meanings, and every other encoded value is used for ordinary normalized numbers as described above. The two values set aside are the smallest possible encoded field, 0 (all-zero), and the largest possible encoded field, $2^e - 1$ (all-one), where e is the number of exponent bits. Crossing these two reserved exponent codes (all-zero, all-one) against the two possibilities for the significand (all-zero, non-zero) gives four cases:

- **All-zero field, all-zero significand** $\Rightarrow$ the value is ± 0 .
- **All-zero field, non-zero significand** $\Rightarrow$ the value is a *sub-normal* (denormal) number. For this case, the standard's rule is: use $E_{\min}$ as the exponent directly, instead of computing it from the field the usual way. The leading significand bit is also taken to be 0 instead of 1. This lets the format represent values smaller than the smallest normal number, at reduced precision, instead of jumping abruptly to zero — *gradual underflow*.
- **All-one field, all-zero significand** $\Rightarrow$ the value is $\pm\infty$.
- **All-one field, non-zero significand** $\Rightarrow$ the value is *NaN* (Not-a-Number), used for results such as $0/0$, $\infty - \infty$, or $\sqrt{-1}$. The sign bit is still present in this encoding, but it carries no meaning for NaN — there is no separate $+\text{NaN}$ or $-\text{NaN}$, just NaN.

Because two encoded exponent values are reserved this way, a format with an e -bit exponent field has only $2^e - 2$ encoded patterns left for normal numbers, giving an effective normal exponent range of $E \in [-(2^{e-1} - 2), 2^{e-1} - 1]$ once IEEE 754's standard bias, $2^{e-1} - 1$, is applied (Section 3.2 gives the general encoding rule this bias is used in). Overflow — a true mathematical result larger than the largest representable normal number — is handled by rounding to $\pm\infty$ (round-to-nearest modes) or by saturating at the largest finite representable value (some low-precision AI formats, discussed in Section 7, choose exactly this alternative to avoid ever producing an infinity).

2.5 Terminology: Precision, Machine Epsilon, and ULP

A handful of terms recur throughout the rest of this paper.

- **Precision (P).** The number of radix digits used for the significand. For a normalized binary format with an implicit leading bit and m stored fraction bits, $P = m + 1$.
- **Machine Epsilon (ε).** The gap between 1 and the next larger representable floating-point number. 1 itself is represented as $1.\underbrace{00\cdots0}_{P-1} \times 2^0$ — all $P - 1$ fraction bits zero. The next representable value increases only the least significant fraction bit, giving $(1 + 2^{-(P-1)}) \times 2^0$. Subtracting the two gives the gap directly: $\varepsilon = 2^{-(P-1)}$. In the toy format above, $P = 3$ gives $\varepsilon = 2^{-2} = 0.25$.
- **Unit in the Last Place (ULP).** For a real number $x \in \mathbb{R}$, $\text{ulp}(x)$ is the gap between the two representable floating-point numbers $\text{FP}_1, \text{FP}_2 \in \mathbb{R}'$ that lie closest around x — even if x itself equals one of them — expressed as a multiple of ε scaled by x 's exponent. In the toy format, $\text{ulp}(1) = 1\varepsilon$, $\text{ulp}(0.6) = \frac{1}{2}\varepsilon$ (because 0.6 falls in the $E = -1$ band, where consecutive representable values are half as far apart as they are at $E = 0$), and $\text{ulp}(2.1) = 2\varepsilon$ (the $E = 1$ band). ULP is the natural unit for measuring rounding error: in the worst case, the true real number x falls exactly halfway between FP_1 and FP_2 , so rounding to whichever one is nearer moves the result by at most half their gap. This is why a correctly rounded result should differ from the true mathematical number by at most $\frac{1}{2} \text{ulp}$.

Precision vs. accuracy.

P , ε , and ulp now give us everything needed to say precisely what *precision* and *accuracy* each mean — two words that get used as if they were interchangeable, but answer different questions.

Precision is exactly the P defined above: a count of digits, not a size. This is the same idea as “significant figures” in ordinary scientific notation: 1.23×10^5 and 1.23×10^{-5} both have 3 significant digits, even though one number is ten billion times bigger than the other. P works the same way — it is fixed once for the whole format, and every representable value, in every exponent band, is given the same P digits to work with, regardless of how big or small that value is.

Now, because P (the digit count) stays fixed while the exponent changes, the same P digits buy a bigger or smaller *absolute* step depending on the exponent band — and that absolute step is exactly $\text{ulp}(x)$ from above. In the toy format, $P = 3$ in every band, yet that fixed digit budget buys a step of only 0.125 in the $E = -1$ band but a step of 0.25 in the $E = 0$ band. So precision (P) is the cause; the size of $\text{ulp}(x)$ in any given band is one effect of it.

Accuracy asks a different question entirely, one that has nothing to do with the format's design and everything to do with one specific number: once you actually round a real number x to its nearest representable value, how far off is the result? Take two real numbers that both fall in the toy format's $E = -1$ band, where $\text{ulp} = 0.125$ for both of them: $x = 0.625$ is itself representable, so it is stored with 0 error — perfectly accurate. $x = 0.6$ is not representable, so it rounds to the nearest entry, 0.625, an error of 0.025 — less accurate, even though it sits in the exact same exponent band, governed by the exact same P and the exact same ulp .

That is the whole distinction: precision (P) is a fixed budget of digits the format hands out equally everywhere; accuracy is how far one particular rounding of one particular number actually lands from the truth, given that budget. Precision limits how bad accuracy can possibly get in a single rounding — never worse than $\frac{1}{2} \text{ulp}$, as shown above — but it does not fix accuracy to one value, since where exactly x falls within its ulp -wide gap still decides the actual error.

The 0.625 vs. 0.6 example above picks two specific numbers to make the contrast vivid, but it is worth stating the general pattern behind it plainly, because this paper keeps coming back to it: across the whole continuum of real numbers — not just the rare ones that happen to land exactly on a representable value — increasing precision increases accuracy, and decreasing precision decreases accuracy. Raising P shrinks ε , which shrinks $\text{ulp}(x)$ in every band; since almost every real number falls strictly between two representable values rather than exactly on one, a smaller $\text{ulp}(x)$ means a smaller rounding error for essentially every x . Lowering P does the reverse. This is exactly the trade-off Section 4 onward keeps returning to: every low-precision AI format surveyed later in this paper buys memory, bandwidth, and compute savings by lowering P , and the direct cost of that choice, across the whole range of values the format represents, is lower accuracy.

With these mechanics and this vocabulary in hand, Section 3 turns to how IEEE 754 actually fixes β , P , $E_{\min}$, and $E_{\max}$ for its standard formats, starting with 32-bit single precision.

3 The IEEE 754 Standard

3.1 Historical Evolution

Table 2 summarizes how the standard now known as IEEE 754 arrived at its current form.

Table 2: Evolution of the IEEE floating-point standards.

Year	Milestone
1985	IEEE Std 754-1985. The original binary ($\beta = 2$) floating-point standard. Defined single-precision (32-bit), double-precision (64-bit), and an extended-precision (80-bit, as used by x87) format [5].
1987	IEEE Std 854-1987. A companion, radix-independent standard supporting both $\beta = 2$ and $\beta = 10$ (decimal) arithmetic [6].
2008	IEEE Std 754-2008. Merged 754-1985 and 854-1987 into a single standard, extensively revised the binary format definitions, and added half-precision (16-bit), quadruple-precision (128-bit), and an octuple-precision (256-bit) format alongside decimal formats.
2019	IEEE Std 754-2019. A bug-fix-and-clarification revision of 754-2008, with no new formats [7].
Ongoing	IEEE P3109. A working group formed under the IEEE Standards Association to define a binary arithmetic and data format standard specifically for machine-learning-optimized hardware — targeting 8-bit and narrower formats, where the traditional 754 design goals (maximal precision for general scientific computing) give way to ML-specific goals such as dynamic range under quantization and hardware efficiency. The effort is being pursued as a standard adjunct to, not a replacement for, IEEE 754-2019 [1, 8]. See Section 14.1 for further discussion.

3.2 General IEEE Binary Encoding

Every IEEE binary floating-point format packs three fields into a fixed number of bits: a 1-bit sign, an e -bit exponent field, and an m -bit fraction (significand) field (giving precision $P = m + 1$ once the implicit leading bit is counted), laid out from most significant to least significant bit as *sign* / *exponent* / *fraction* — the same left-to-right order used in Figure 2 and every other bit-layout figure in this paper. All of the formats in this paper follow the conventions introduced in Section 2.4:

- As introduced in Section 2.4, the exponent is stored as an encoded field, $E + \text{bias}$, rather than the signed value E directly. IEEE 754 fixes the bias for an e -bit exponent field at $2^{e-1} - 1$.
- An all-zero exponent field with:
 - an all-zero fraction represents ± 0 .
 - a non-zero fraction represents a sub-normal number (implicit leading bit 0, exponent pinned at $E_{\min}$).
- An all-one exponent field with:
 - an all-zero fraction represents $\pm \infty$.
 - a non-zero fraction represents NaN.

Table 3 lists the standard IEEE 754-2019 binary formats. Single precision (`float32`) is worked through in full detail below, as the reference point for every low-precision format later in this survey.

Table 3: IEEE 754-2019 standard binary formats.

Format	Total bits	Sign	Exponent	Fraction	Bias
binary16 (half)	16	1	5	10	15
binary32 (single)	32	1	8	23	127
binary64 (double)	64	1	11	52	1023
binary128 (quad)	128	1	15	112	16383
binary256 (octuple)	256	1	19	236	262143

3.3 IEEE Single Precision (`float32`) in Detail

Figure 2 shows the 32-bit layout: 1 sign bit, an 8-bit biased exponent (bits 30–23), and a 23-bit fraction (bits 22–0).

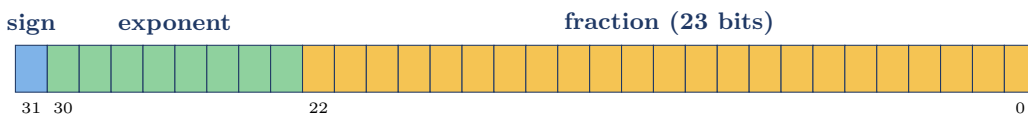


Figure 2: IEEE float32 (single precision) bit layout. The leading significand bit is implicit (assumed 1 for normal numbers, 0 for sub-normal numbers) and is therefore not stored.

- Bits 0–22 (23 bits) form the fraction. Together with the implicit leading bit, the effective precision is $P = 24$ bits.
- Bits 23–30 (8 bits) form the biased exponent, with bias 127: the stored field value E_{field} encodes the true exponent $E = E_{\text{field}} - 127$. Since $E_{\text{field}} = 0$ and $E_{\text{field}} = 255$ are reserved (Section 2.4), normal numbers span $E_{\text{field}} \in [1, 254]$, i.e. true exponents $E \in [-126, 127]$.
- Bit 31 is the sign bit (0 = positive, 1 = negative).

Normal numbers.

For $1 \leq E_{\text{field}} \leq 254$, the represented value is

$$\text{value} = (-1)^{\text{sign}} \times 2^{E_{\text{field}} - 127} \times 1.b_{22}b_{21} \cdots b_0 \quad (2)$$

Table 4 shows the extremes of the normal range, and Figure 3 places those extremes on the real line.

Table 4: IEEE float32 normal-number extremes.

Sign	Exponent field	Fraction	Value
0	00000001 (−126)	00...0	$+2^{-126} \approx 1.18 \times 10^{-38}$ (smallest positive normal, i.e. closest to zero)
0	11111110 (+127)	11...1	$+(2 - 2^{-23}) \times 2^{127} \approx 3.4 \times 10^{38}$ (largest normal / most positive)
1	00000001 (−126)	00...0	$-2^{-126} \approx -1.18 \times 10^{-38}$ (largest negative normal, i.e. closest to zero)
1	11111110 (+127)	11...1	$-(2 - 2^{-23}) \times 2^{127} \approx -3.4 \times 10^{38}$ (smallest normal / most negative)

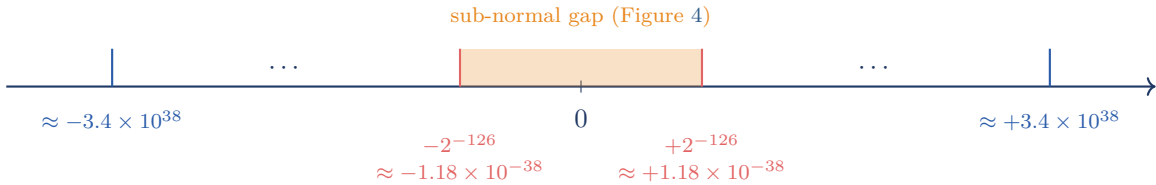


Figure 3: Where float32’s normal numbers sit on the real line (schematic, not to scale). Normal numbers exist only outside the interval $(-2^{-126}, 2^{-126})$; that gap is filled by sub-normals (Figure 4). Moving outward from $\pm 2^{-126}$, normal numbers continue up to $\pm(2 - 2^{-23}) \times 2^{127} \approx \pm 3.4 \times 10^{38}$, with the gap between consecutive representable values growing as magnitude grows — the same clustering behavior already seen in the toy example of Figure 1, just at a far larger scale.

Sub-normal numbers.

For $E_{\text{field}} = 0$ with a non-zero fraction, the implicit leading bit becomes 0 and the exponent is pinned at $E_{\text{min}} = -126$ (not -127 , which would be inconsistent with the bias):

$$\text{value} = (-1)^{\text{sign}} \times 2^{-126} \times 0.b_{22}b_{21} \cdots b_0 \quad (3)$$

This lets float32 represent values as small as $2^{-126} \times 2^{-23} = 2^{-149} \approx 1.4 \times 10^{-45}$, at successively reduced precision as magnitude shrinks toward zero — *gradual underflow*, rather than an abrupt

jump to zero once the smallest normal value is passed. Figure 4 zooms into exactly the gap left blank in Figure 3 and shows how sub-normals fill it.

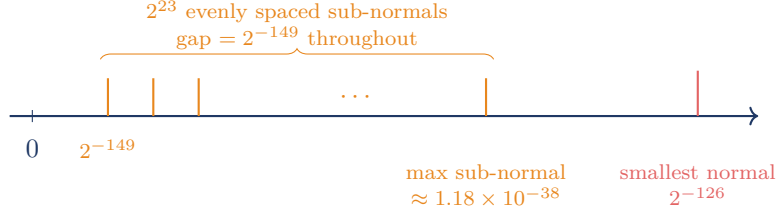


Figure 4: Zooming into the gap $[0, 2^{-126})$ left blank in Figure 3: sub-normals fill it with 2^{23} evenly spaced values, each 2^{-149} apart, running from 0 up to just below the smallest normal number 2^{-126} . Unlike normal numbers, whose spacing doubles with every increase in exponent (Figure 3), sub-normal spacing is uniform across the entire sub-normal range — exactly what makes underflow *gradual* rather than abrupt. The negative side, $(-2^{-126}, 0]$, is the mirror image about 0.

Special numbers.

Table 5 summarizes the remaining reserved encodings.

Table 5: IEEE float32 special-value encodings.

Sign	Exponent field	Fraction	Represents
0	00000000 (-127)	0	$+0$
1	00000000 (-127)	0	-0
0	11111111 ($+128$)	0	$+\infty$
1	11111111 ($+128$)	0	$-\infty$
X	11111111 ($+128$)	non-zero	NaN

The same three-part structure — normal / sub-normal / special, driven entirely by the two reserved exponent-field codes — governs every other IEEE binary format (binary16, binary64, ...) simply by substituting that format’s own field widths and bias, and it is also, with small but important deviations discussed in Sections 6 and 7, the structure that the low-precision AI formats surveyed later in this paper build on.

3.4 Rounding

IEEE 754 defines several rounding attributes, or “rounding modes,” that determine how a real-valued result that is not exactly representable gets mapped onto the format’s finite set of representable values.

Round-to-nearest modes.

The default and by far the most commonly used mode is *round-to-nearest, ties-to-even* (RNE): round to the closer of the two neighboring representable values, and in the case of an exact tie, round to whichever of the two has a fraction field ending in 0 (i.e. is “even”), which avoids a statistical bias that a fixed round-half-up rule would introduce over many operations. IEEE 754 also defines a second, optional round-to-nearest mode, *ties-to-away* (RNA): on an exact tie, round to whichever of the two neighbors is farther from zero instead of whichever is even. RNA

is simpler to reason about than RNE but reintroduces a small statistical bias, so RNE remains the required default for binary formats.

Directed rounding modes.

The standard also defines three directed rounding modes, used less often in general computation but relevant to interval arithmetic and to some of the low-precision conversion rules discussed in Section 7:

- **Round toward $+\infty$** (round up): always round to the nearest representable value that is greater than or equal to the true real number.
- **Round toward $-\infty$** (round down): always round to the nearest representable value that is less than or equal to the true real number.
- **Round toward zero** (truncation): always round to the nearest representable value whose magnitude is no greater than the true real number’s magnitude, i.e. simply drop the extra bits.

Stochastic rounding.

AI hardware and low-precision training software commonly use one more rounding mode that is not part of the IEEE 754 standard itself: *stochastic rounding* (SR). Round-to-nearest always rounds a given value x to the same one of its two representable neighbors $\text{FP}_1 < x < \text{FP}_2$. Stochastic rounding instead makes this choice randomly, using a two-step procedure repeated fresh every single time a value is rounded:

1. **Compute a probability from x ’s position.** Compute

$$p = \Pr[\text{round}(x) = \text{FP}_2] = \frac{x - \text{FP}_1}{\text{FP}_2 - \text{FP}_1} \quad (4)$$

This p is just a number between 0 and 1 that says how close x already is to FP_2 : p is close to 1 if x is almost at FP_2 , close to 0 if x is almost at FP_1 , and exactly 0.5 if x sits exactly halfway between them. On its own, this step does not round anything — it only fixes the odds for the step below.

2. **Draw a random number and compare it to p .** Draw a random number u , uniformly at random between 0 and 1 (think of spinning a dial that is equally likely to stop anywhere from 0 to 1). Then:

if $u < p$: round x up to FP_2 , otherwise: round x down to FP_1 .

Because u is drawn fresh every time, rounding the same x twice can give two different answers — unlike round-to-nearest, which always rounds the same x to the same neighbor. What Equation (4) guarantees is only a long-run property: if this two-step procedure is repeated on the same x many times, the fraction of those repeats that land on FP_2 converges to p , and consequently the average of all the rounded results converges to x itself. This is what it means to say stochastic rounding is *unbiased*: $\mathbb{E}[\text{round}(x)] = x$, where $\mathbb{E}[\cdot]$ denotes this long-run average, not the result of any single rounding.

Round-to-nearest, by contrast, rounds the same input the same way every time, so if the same rounding decision is repeated many times over — as happens when repeatedly accumulating small gradient updates into a low-precision weight during training — the rounding error can keep pointing in the same direction and add up instead of canceling out. Stochastic rounding avoids this by letting the error average out to zero over many roundings. This is why stochastic

rounding shows up specifically in the AI hardware and low-precision training formats covered later in this survey (Sections 6 and 7), even though it plays no role in general-purpose IEEE 754 arithmetic.

4 Low-Precision Floating-Point Formats

A *low-precision* floating-point format is simply any format narrower than the 32-bit single precision format of Section 3 — built from the same sign / exponent / fraction structure, just with fewer bits handed to the exponent field, the fraction field, or both. Shrinking the exponent field narrows the dynamic range (the gap between the smallest and largest representable magnitudes); shrinking the fraction field coarsens precision (the gap between consecutive representable values gets bigger, so more real numbers get rounded to the same stored value). The rest of this paper surveys the specific low-precision formats that AI hardware actually uses — 16-bit formats (Section 5), 8-bit formats (Section 6), and sub-byte 6-bit and 4-bit formats with blockwise scaling (Sections 7, 9 and 10). Before going through them one by one, it is worth asking why the field moved to narrower formats in the first place.

Why Low Precision? Motivation from AI/ML Workloads.

Deep neural network training and inference consist overwhelmingly of matrix multiplications and convolutions, both reducible to enormous numbers of multiply-accumulate operations over tensors that can run into the hundreds of billions of elements for today’s large language models. Three separate cost dimensions all improve, roughly linearly, as the bit width of a stored value shrinks:

- **Memory footprint.** Halving tensor element width halves the memory needed to hold weights, activations, gradients, and optimizer state, which directly determines how large a model can fit on a given accelerator or cluster.
- **Memory bandwidth.** Moving data between an accelerator’s HBM memory and its on-chip SRAM, and between accelerators, is frequently the actual bottleneck in training and (especially) inference throughput; narrower formats move proportionally less data for the same tensor.
- **Compute density.** A narrower multiply-accumulate unit occupies less silicon area and consumes less energy per operation, so a fixed die budget can pack proportionally more of them, and a fixed power budget can run proportionally more of them per second.

The catch, of course, is accuracy: a narrower significand means a smaller P , and Section 2.5 already established what that does across the whole continuum of values — lower precision means lower accuracy, i.e. coarser rounding. A narrower exponent, separately, means a smaller dynamic range before overflow or underflow. The entire history traced in this paper — from 32-bit single precision down through 16-bit formats, 8-bit formats, and finally sub-byte 6-bit and 4-bit formats with blockwise scaling — is a history of the field finding, empirically and then architecturally, just how much precision and range a given stage of training or inference actually needs, and building hardware that stops paying for anything beyond that. Micikevicius et al. [16] were among the first to show that training could tolerate 16-bit compute broadly, as long as a full-precision (FP32) copy of the weights was retained for the accumulation and update step; Gholami et al. [3] survey the subsequent, much larger body of work on quantizing trained networks directly to 8 bits and below for inference. Every format in the rest of this paper sits somewhere on that same range/precision/bit-width trade-off curve, tuned for one part of the training or inference pipeline.

5 16-bit and 19-bit Formats: The First Step Down

The three formats surveyed in this section are all used heavily in AI hardware today, but only two of them were actually designed for AI. IEEE float16 predates deep learning: it originated in computer graphics in the late 1990s and early 2000s and was folded into IEEE 754-2008 years before AI workloads made any use of it; it was only later repurposed for AI training once GPUs gained hardware support for it. bfloat16 and TF32, by contrast, were purpose-built for AI workloads from the outset. The three also differ in how far they shrink from IEEE single precision (float32’s 32 bits): float16 and bfloat16 both drop all the way to 16 bits, simply rebalancing how those bits are split between the exponent and fraction fields, while NVIDIA’s TF32 trims more modestly, to 19 bits.

5.1 IEEE Half Precision (float16)

As noted above, float16 was not designed with AI in mind: it started out in computer graphics (used informally for years before that in tools such as ILM’s OpenEXR format and Nvidia’s Cg shading language) and was only later added to the IEEE 754 standard in the 2008 revision, well before deep learning made any use of it.

IEEE 754 defines this 16-bit format (Table 3): 1 sign bit, 5 exponent bits (bias 15), and 10 fraction bits, giving an effective precision of 11 bits (Figure 5).



Figure 5: IEEE float16 (half precision) bit layout.

With only 5 exponent bits, the normal-number exponent range is $E \in [-14, 15]$, giving a magnitude range of roughly 6.1×10^{-5} to 6.5×10^4 . This range is narrow enough that half-precision training runs frequently overflow or underflow activations and gradients unless carefully scaled — one reason later formats deliberately reallocate bits from fraction to exponent.

5.2 Google’s bfloat16

bfloat16 (“Brain Floating Point,” after Google Brain) is also 16 bits wide, but keeps the full 8-bit exponent field of float32 (same bias, 127) and shrinks the fraction to 7 bits instead (Figure 6) [45].

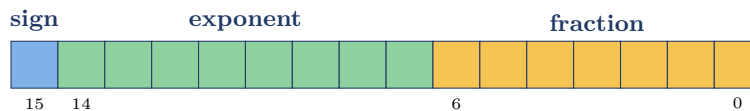


Figure 6: bfloat16 bit layout — same exponent width and bias as IEEE float32, fraction truncated to 7 bits.

Because the exponent field is unchanged from float32, bfloat16 has (very nearly) the same representable *range* as full single precision, $\approx 1.2 \times 10^{-38}$ to $\approx 3.4 \times 10^{38}$, at the cost of much coarser precision (an effective 8-bit significand versus float32’s 24). This is a deliberate trade: it is far easier to convert a float32 value to bfloat16 and back (truncate or round the fraction; exponent is untouched) than to a genuine IEEE float16 value, and it makes bfloat16 training numerically much more suitable for the wide dynamic range that gradients and activations can take on, at the cost of keeping fewer fraction bits.

5.3 NVIDIA’s TensorFloat-32 (TF32)

TensorFloat-32 is NVIDIA’s tensor-core-internal compute format, introduced with the Ampere architecture [23]. It uses 19 bits total: the same 8-bit exponent field (and bias) as float32 and bfloat16, but a 10-bit fraction — matching float16’s fraction width (Figure 7).

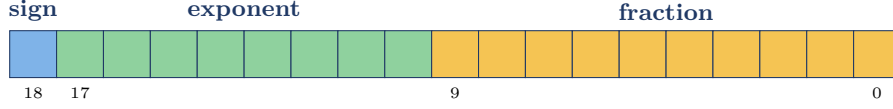


Figure 7: NVIDIA TensorFloat-32 bit layout. TF32 is a Tensor Core compute format, not a memory storage format: tensors are still stored as FP32 in memory, and the GPU truncates operands to TF32 only for the matrix multiply itself, accumulating in full FP32.

TF32 therefore has the same wide dynamic range as bfloat16 and float32 (full 8-bit exponent), while recovering some of the precision bfloat16 gives up (10 fraction bits instead of 7). In practice, TF32 is used transparently inside the Tensor Core: application code still stores and manipulates ordinary FP32 tensors, and the hardware truncates operands to TF32 internally for the multiply, accumulating the products in full FP32 — a middle ground that recovers much of FP32’s numerical behavior at a substantial fraction of lower-precision throughput.

5.4 Comparison

Table 6 summarizes the three formats side by side.

Table 6: Comparison of 16-/19-bit AI compute formats against IEEE float32 and float16.

Format	Bits	Exp.	Frac.	Eff. precision	Range vs. FP32
float32 (FP32)	32	8	23	24 bits	reference range
float16 (FP16)	16	5	10	11 bits	much narrower ($\sim 6 \times 10^{-5} - 6.5 \times 10^4$)
bfloat16 (BF16)	16	8	7	8 bits	<i>same as FP32</i>
TensorFloat-32 (TF32)	19	8	10	11 bits	<i>same as FP32</i>

The pattern that emerges — and that repeats at every smaller bit width discussed later in this paper — is a direct trade-off between range (controlled by exponent width) and precision (controlled by fraction width) at a fixed total bit budget. FP16 spends its 16 bits on precision at range’s expense; bfloat16 spends the same 16 bits on range at precision’s expense; TF32 spends 3 additional bits to get most of both.

6 8-bit Floating-Point Formats (FP8)

FP8 is the first format in this survey to depart from IEEE 754’s normal / sub-normal / special structure in a way that matters: at only 8 total bits, giving up even one bit pattern to a rarely-used special value is comparatively expensive, so the FP8 formats used in AI hardware make deliberate, format-specific trade-offs in how (or even whether) they represent some of the special numbers.

6.1 The Two FP8 Encodings: E5M2 and E4M3

FP8 is standardized, in the sense of common industry practice (formalized jointly by NVIDIA, Arm, and Intel, and now the starting point for the IEEE P3109 effort discussed in Section 14.1), as two sibling encodings sharing the same 1-bit sign field but splitting the remaining 7 bits differently between exponent and fraction [17]:

- **E5M2** (5 exponent bits, 2 fraction bits, also called FP8-152): wider range, less precision.
- **E4M3** (4 exponent bits, 3 fraction bits, also called FP8-143): narrower range, more precision.

Table 7 lays out this taxonomy; Section 7 gives the same taxonomy for the narrower FP6 and FP4 types.

Table 7: Taxonomy of the FP8 floating-point types.

Category	Type	Exp. bits	Mant. bits	Key property
FP8 (8 bits)	E4M3	4	3	more precision
	E5M2	5	2	more range



Figure 8: The two FP8 bit layouts.

Of the two formats, only E4M3 codes a few bit patterns differently from IEEE 754. E5M2 follows the general encoding rule from Section 3.2 everywhere, with no special-casing at all: it is simply that rule applied with 5 exponent bits and 2 fraction bits, the same way IEEE float16 applies it with 5 exponent bits and 10 fraction bits.

E4M3 still follows the IEEE rule for the all-zero exponent code: it gives zero and sub-normal numbers the standard way, exactly like E5M2. It departs from IEEE only for the all-one exponent code, the one reserved for special values. Ignoring the sign bit, which plays no role in this deviation, the all-one exponent code covers two kinds of bit patterns, and E4M3 codes each of them differently from IEEE:

- **All-one exponent, all-zero fraction** — the pattern IEEE reserves for $\pm\infty$: E4M3 does not represent infinity at all, so this pattern is instead decoded as an ordinary finite normal number, using the same formula as any other exponent code (its magnitude works out to 256, well inside E4M3’s normal range of up to ± 448).
- **All-one exponent, non-zero fraction** — the patterns IEEE reserves for NaN: E4M3 reserves only one of these seven patterns, the one with every fraction bit set ($S.1111.111_2$), for NaN. The other six are likewise decoded as ordinary finite normal numbers.

This is the trade-off E4M3 makes at only 8 total bits: a full IEEE-style reservation would cost 8 of its bit patterns per sign (1 for infinity plus 7 for NaN); E4M3 instead keeps 7 of those 8 as ordinary finite normal numbers, and reserves only the last one for NaN. Table 8 gives the resulting encoding details.

Table 8: FP8 encoding details for E4M3 and E5M2.

	E4M3	E5M2
Exponent bias	7	15
Infinities	N/A (not representable)	$S.11111.00_2$
NaN	$S.11111.111_2$ (one pattern)	$S.11111.\{01, 10, 11\}_2$
Zeros	$S.00000.000_2$	$S.000000.00_2$
Max normal	$S.11111.110_2 = \pm 1.75 \times 2^8 = \pm 448$	$S.111110.11_2 = \pm 1.75 \times 2^{15} = \pm 57,344$
Min normal	$S.00001.000_2 = \pm 2^{-6}$	$S.000001.00_2 = \pm 2^{-14}$
Max sub-normal	$S.00000.111_2 = \pm 0.875 \times 2^{-6}$	$S.000000.11_2 = \pm 0.75 \times 2^{-14}$
Min sub-normal	$S.00000.001_2 = \pm 2^{-9}$	$S.000000.01_2 = \pm 2^{-16}$

6.2 How FP8 Is Used in Deep Learning

Empirically, the two FP8 encodings turn out to suit different phases of neural network training [17]:

- In the **forward pass**, weights and activations tend to need more *precision* than range, favoring E4M3.
- In the **backward pass**, gradients can span a much wider *range* of magnitudes, favoring E5M2.

Converting a higher-precision tensor down to FP8 routinely risks overflow or underflow given FP8’s narrow dynamic range, so production FP8 pipelines apply scaling to stay inside FP8’s representable range, exactly as Section 8 describes in general. For FP8, scaling is still an optional add-on; for every sub-8-bit format discussed later in this paper, it becomes the entire point of the format.

7 Sub-Byte Floating-Point Formats: FP6 and FP4

Pushing below 8 bits, the industry has converged on two further families of *low-precision (LP) types*: 6-bit FP6 and 4-bit FP4, each again offered as a precision-favoring and a range-favoring variant, exactly as FP8 was. Table 9 lays out their taxonomy.

Table 9: Taxonomy of the sub-byte low-precision (LP) floating-point types: FP6 and FP4.

Category	Type	Exp. bits	Mant. bits	Key property
FP6 (6 bits)	E2M3	2	3	more precision
	E3M2	3	2	more range
FP4 (4 bits)	E2M1	2	1	only variant in production use

7.1 Shared Encoding Rules

The sub-byte low precision types reuse the general IEEE encoding convention from Section 3.2 — biased exponent, all-zero exponent for zero/sub-normal, all-one exponent for infinity/NaN — *where bit budget allows*. As already seen with E4M3 in Section 6.1, however, the tighter the bit budget, the more of that convention gets sacrificed for extra representable range or precision: as shown below, neither FP6 variant nor the FP4 format represents infinity or NaN at all, reserving *no* encoding for either and instead using their full encoding space for finite values.

7.2 FP6: E2M3 and E3M2

Table 10 gives the encoding details for the two FP6 variants. Neither represents infinity or NaN.

Table 10: FP6 encoding details.

	E2M3	E3M2
Exponent bias	1	3
Infinities	N/A	N/A
NaN	N/A	N/A
Zeros	$S.00.000_2$	$S.000.00_2$
Max normal	$S.11.111_2 = \pm 1.875 \times 2^2 = \pm 7.5$	$S.111.11_2 = \pm 1.75 \times 2^4 = \pm 28$
Min normal	$S.01.000_2 = \pm 1.0 \times 2^0 = \pm 1.0$	$S.001.00_2 = \pm 1.0 \times 2^{-2} = \pm 0.25$
Max sub-normal	$S.00.111_2 = \pm 0.875 \times 2^0 = \pm 0.875$	$S.000.11_2 = \pm 0.75 \times 2^{-2} = \pm 0.1875$
Min sub-normal	$S.00.001_2 = \pm 0.125 \times 2^0 = \pm 0.125$	$S.000.01_2 = \pm 0.25 \times 2^{-2} = \pm 0.0625$



Figure 9: FP6 bit layouts.

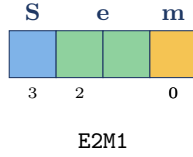
7.3 FP4: E2M1

At 4 bits, the trade-off is at its most extreme. Table 11 gives the details for E2M1, the FP4 format used in production.

Table 11: FP4 (E2M1) encoding details.

E2M1	
Exponent bias	1
Infinities	N/A
NaN	N/A
Zeros	$S.00.0_2$
Max normal	$S.11.1_2 = \pm 1.5 \times 2^2 = \pm 6.0$
Min normal	$S.01.0_2 = \pm 1.0 \times 2^0 = \pm 1.0$
Max sub-normal	$S.00.1_2 = \pm 0.5 \times 2^0 = \pm 0.5$
Min sub-normal	$S.00.1_2 = \pm 0.5 \times 2^0 = \pm 0.5$

Table 11’s “Max sub-normal” and “Min sub-normal” rows show the same value, ± 0.5 , and this is not a mistake: with only a 1-bit mantissa field, E2M1 has exactly one non-zero sub-normal bit pattern, so there is nothing for a maximum and a minimum to differ over.

**Figure 10:** FP4 (E2M1) bit layout. E2M1 is the format used in production (MXFP4, NVFP4; Sections 9 and 10).

As shown in Table 12 below, E2M1 has only 16 total bit patterns, and its positive, finite, non-zero values are exactly $\{0.5, 1, 1.5, 2, 3, 4, 6\}$ — with a gap between 0 and 0.5 that is the same size as the gap between 0.5 and 1, but a gap between 4 and 6 that is twice the gap between 3 and 4. This coarseness is precisely why FP4 is unusable directly on real neural-network tensors without the scaling machinery of Section 9: a single shared exponent field with only 2 bits gives nowhere near enough per-value dynamic range on its own.

Table 12: All 16 E2M1 (FP4) bit patterns and their represented values.

Bit pattern	Value	Kind
0.00.0 ₂	0	zero
0.00.1 ₂	0.5	sub-normal
0.01.0 ₂	1	normal
0.01.1 ₂	1.5	normal
0.10.0 ₂	2	normal
0.10.1 ₂	3	normal
0.11.0 ₂	4	normal
0.11.1 ₂	6	normal
1.00.0 ₂	−0	zero
1.00.1 ₂	−0.5	sub-normal
1.01.0 ₂	−1	normal
1.01.1 ₂	−1.5	normal
1.10.0 ₂	−2	normal
1.10.1 ₂	−3	normal
1.11.0 ₂	−4	normal
1.11.1 ₂	−6	normal

8 The Idea of Scaling

At this point, it helps to draw a clean line between two groups of formats already covered in this paper, based on total bit width. `float32` (Section 3), `float16`, `bfloat16`, and `TF32` (Section 5) all spend 16 bits or more per value; this section calls them the *high-precision* types. `FP8` (Section 6), `FP6`, and `FP4` (Section 7) spend 8 bits or fewer; this section calls them the *low-precision* types.

Why does this split matter? Because AI workloads routinely need to convert a tensor from a high-precision format down to a low-precision one, to get the memory, bandwidth, and compute savings Section 4 already laid out — and converting directly, with no adjustment, is risky precisely because low-precision types pack so little exponent range into so few bits. Section 7 already gave a concrete illustration: `E2M1`, `FP4`’s only production variant, has just 2 exponent bits, and its representable magnitudes stop at 6 on the high end and 0.5 on the low end (ignoring zero). Directly converting a high-precision tensor down to a low-precision type this narrow routinely pushes values outside that representable range: large values overflow, and small values underflow to zero. Neither outcome is acceptable for AI workloads, where both large-magnitude activations and small-magnitude gradients need to survive the conversion.

What is needed, then, is some mechanism that makes overflow and underflow the rare exception

rather than the routine outcome. That mechanism is *scaling*.

The idea involves two distinct steps, and it is worth being precise about what each one does. *Down-scaling* divides every high-precision value by a shared scale factor first; *down-conversion* then rounds each scaled result onto the nearest value the low-precision format can actually represent. Up-converting reverses both steps in the opposite order: first up-converting the stored low-precision value back to high precision, and only then multiplying it by that same scale factor to recover the original magnitude.

These two steps do different jobs, and neither one replaces the other.

Down-conversion has to happen either way. A low-precision format only has a handful of representable values. So any value going into it, scaled or not, still needs to be rounded onto that small set of values — this is just as true for a plain, unscaled float-to-float conversion.

Down-scaling fixes a different problem: some values do not fit in the low-precision format’s range at all, no matter how they are rounded. Here is a concrete example. Say a low-precision format’s largest representable value is 6, and one of the values to be converted is 500. No rounding rule can help: 500 is nowhere near anything this format can hold, so it simply overflows. Dividing 500 by a scale factor first, say 128, brings it down to about 3.9 — a value well inside the format’s range, which down-conversion can then round normally (to 4, say). This is exactly what down-scaling does for a whole group of values at once: it picks one shared scale factor, sized to that group’s largest value, and divides every value in the group by it before down-conversion rounds each one.

This does not work perfectly for every value in the group, though. One scale factor is shared by the whole group, but the values inside a group do not all have the same magnitude. A scale factor sized to the group’s largest value can still leave some of the group’s smaller values too small to survive, or, in rare cases, leave even the largest value just outside the format’s range after scaling (Section 9.6 covers what happens then). So down-scaling makes overflow and underflow rare, not impossible.

What differs, in practice, is exactly how that scale factor is computed, and how many values of a tensor share one scale factor at a time — an entire tensor, or a much smaller group of values within it. Section 9 covers the Open Compute Project’s Microscaling (MX) format family, the industry-wide standard for this fine-grained, group-wise scaling; Section 10 then covers NVIDIA’s NVFP4, a refinement of the same idea purpose-built for 4-bit formats.

9 Block Scaling: The Microscaling (MX) Format Family

Section 8 introduced scaling in general: divide before down-converting, multiply back after up-converting. *Microscaling* is the Open Compute Project’s concrete, industry-wide instantiation of that idea, and the specific choice it makes is one of granularity: rather than share one scale factor across an entire tensor, MX lets a small *block* of values share one common, wider-range scale factor. This finer granularity is what makes FP6 and FP4 (and, to a lesser extent, FP8) usable in practice, formalized in the Open Compute Project’s Microscaling Formats (MX) Specification v1.0, co-authored by AMD, Arm, Intel, Meta, Microsoft, NVIDIA, and Qualcomm [31, 32, 42].

9.1 Anatomy of an MX-Compliant Block

An MX-compliant format is characterized by three components: (1) a *scale* data type and its encoding (width w bits), (2) a *private* low-precision element data type and its encoding (width d bits) used for each value in the block, and (3) a *scaling block size* k — the number of private elements that share one scale. Figure 11 shows the resulting layout.

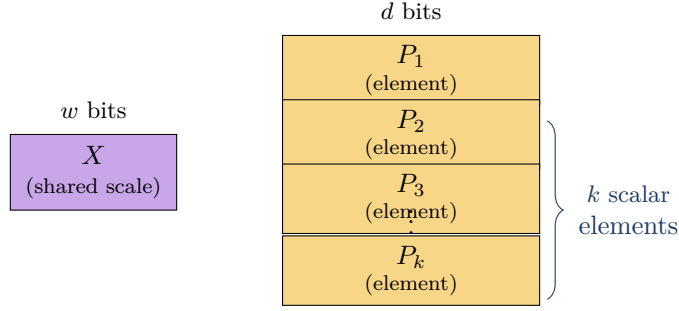


Figure 11: Structure of an MX-compliant block: one shared w -bit scale X applies to k private, d -bit elements $P_1, \dots, P_k$ [31].

9.2 The MXFP Scale Type: E8M0

The shared scale itself needs its own type and encoding. MX defines an 8-bit, exponent-only scale type, E8M0 — an 8-bit unsigned power-of-two exponent with no sign and no mantissa (Table 13), so that scaling a block — dividing by the scale when down-converting, or multiplying by it when up-converting — is always a pure power-of-two operation (a cheap exponent-field add or subtract in hardware, never a general multiply or divide).

Table 13: E8M0 scale-type encoding.

Exponent bias	127
Infinities	N/A
NaN	11111111 ₂
Zeros	N/A
Max normal	11111110 ₂ = 2^{127}
Min normal	00000000 ₂ = 2^{-127}
Sub-normal	N/A

9.3 The MXFP Scaling Block Size: $k = 32$

MX fixes the scaling block size at $k = 32$ for all of its standard formats. Neither the OCP specification nor the paper that introduced microscaling documents a specific reason for choosing 32 over some other value [31, 42]; it is simply given as a parameter. What the numbers do show is the trade-off block size controls. Each block’s shared E8M0 scale costs $8/k$ bits per value on top of the element type’s own d bits, so a larger k spreads that cost thinner: at $k = 32$ it comes to just $8/32 = 0.25$ bits per value. But a larger block also means more elements have to share one scale, so if their magnitudes differ a lot, the smaller ones lose more accuracy. $k = 32$ sits in between: small enough that a block’s values are usually close in magnitude, large enough that the scale’s own overhead stays small. Section 10 shows this trade-off is not fixed: NVFP4 halves the block size to $k = 16$, accepting roughly double the scale overhead, specifically to track magnitude variation more tightly.

9.4 Block Scaling Factor

Given a block of $k = 32$ high-precision values $H = \{h_1, \dots, h_k\}$ and a target low-precision type $LPTtype$, what property should the shared scale X actually have? Two things. First, X has to be a power of two: E8M0 has no mantissa, so it cannot store anything else. Second, dividing the block’s peak magnitude, $\text{amax}(H)$, by X should land as close as possible to $LPTtype$ ’s own largest representable value, without landing past it — that way the block uses as much of $LPTtype$ ’s narrow range as it can, without any element overflowing once it is down-scaled.

That second requirement is harder to satisfy exactly than it sounds: X can only be a power of two, but $\text{amax}(H)$ is essentially never itself an exact power of two, so there is usually no power-of-two X that lands $\text{amax}(H)/X$ exactly on $LPTtype$ ’s largest value — some rounding is unavoidable. The OCP MX specification itself only states the idea in words, and explicitly leaves the exact rounding algorithm out of scope [31]: X should be set to the largest power-of-two less than or equal to the block’s peak magnitude, $\text{amax}(H)$, divided by the largest power-of-two representable in $LPTtype$. Written as a formula, that is

$$X = \frac{\text{largest power of two } \leq \text{amax}(H)}{\text{largest power of two representable in } LPTtype} \quad (5)$$

Equation (5) is still stated in words inside the fraction, not as a closed-form formula. Turning it into a closed-form formula takes four short steps.

Step 1: rewrite the numerator.

The numerator is “largest power of two $\leq \text{amax}(H)$.” To see why $2^{\lfloor \log_2 a \rfloor}$ is exactly that, for any positive real number a , recall what $\log_2 a$ means: it is the power n that 2 would have to be raised to so that $2^n = a$. If a is itself a power of two, say $a = 2^n$, then $\log_2 a = n$ exactly, a whole number. Most values of a are not exact powers of two, though — they fall strictly between two consecutive ones, $2^n \leq a < 2^{n+1}$. Take $a = 6.8$ as a concrete case: it sits between $2^2 = 4$ and $2^3 = 8$, so $\log_2 6.8 \approx 2.77$, a number between 2 and 3 rather than a whole number. The whole-number part of that result, 2, is precisely the exponent of the largest power of two that is still $\leq a$ (here, $2^2 = 4$). Flooring $\log_2 a$ — rounding it down to the nearest integer — is exactly how to extract that whole-number part, so $2^{\lfloor \log_2 a \rfloor}$ gives the largest power of two $\leq a$. Applying this to $a = \text{amax}(H)$, the numerator is $2^{\lfloor \log_2 \text{amax}(H) \rfloor}$.

Step 2: rewrite the denominator.

The denominator is “largest power of two representable in $LPTtype$.” Every low-precision format’s largest normal number has the form $1.xxx \times 2^n$ for some integer n (mantissas of normal numbers always start with 1), and 2^n — the power of two the mantissa multiplies — is itself always representable exactly, with mantissa 1.000... at that same exponent. Since no larger exponent occurs anywhere in the format, 2^n is the largest power of two the format can represent. Call this exponent $n = LPTMaxExp$: the exponent of $LPTtype$ ’s largest normal number. Take E2M1, the 4-bit FP4 format used by MXFP4, as a concrete example: its largest normal value is $1.5 \times 2^2 = 6$ (Table 11), so its exponent is 2, which makes $LPTMaxExp = 2$ for E2M1 — and indeed $2^2 = 4$ is the largest exact power of two E2M1 can represent, even though its largest representable value overall is 6, not 4. With $LPTMaxExp$ defined this way, the denominator is simply $2^{LPTMaxExp}$.

Step 3: substitute and simplify.

Substituting both into Equation (5) gives

$$X = \frac{2^{\lfloor \log_2 \text{amax}(H) \rfloor}}{2^{LPTMaxExp}}.$$

Dividing two powers of the same base subtracts their exponents, so this collapses to

$$X = 2^{\lfloor \log_2 \text{amax}(H) \rfloor - LPTMaxExp}.$$

Step 4: clamp to what E8M0 can store.

E8M0 stores only an 8-bit, bias-127 exponent field, so it can only hold exponents in $[-127, 127]$ (Table 13). Clamping the exponent above to that range gives the closed-form equation actually used in practice,

$$X = 2^{\text{clamp}(\lfloor \log_2 \text{amax}(H) \rfloor - LPTMaxExp, -127, 127)} \quad (6)$$

Problem: Equation (6) can compute a useless scale.

Flooring $\log_2 \text{amax}(H)$ can leave X too small to do its job at all. Here is an example. Take a group of high-precision values whose peak magnitude is $\text{amax}(H) = 6.8$, to be down-converted into E2M1 ($\text{amax}(\text{e2m1}) = 6$). Flooring gives $\lfloor \log_2 6.8 \rfloor = 2$, and E2M1's $LPTMaxExp$ is also 2, so Equation (6) gives

$$X = 2^{\text{clamp}(2-2, -127, 127)} = 2^0 = 1.$$

Dividing by 1 changes nothing — every value in the group is left exactly as it was, so no scaling actually happens. The group's peak value, 6.8, is still past E2M1's range, exactly as it would have been with no scaling at all, so it has to fall back on the saturation convention of Section 9.6 instead of being scaled into range cleanly. In this case, Equation (6) fails at the one job scaling exists to do.

NVIDIA's fix.

NVIDIA's own MXFP8 implementation, documented in its Transformer Engine software [28], closes this gap with two changes to Equation (6).

Change 1: stop throwing away where $\text{amax}(H)$ sits. Equation (6) floors $\log_2 \text{amax}(H)$ before doing anything else. Flooring only keeps which two consecutive powers of two $\text{amax}(H)$ falls between — for 6.8, that is $2^2 = 4$ and $2^3 = 8$ — and throws away exactly where in between them it sits. That is the crucial information missing from Equation (6): 6.8 happens to sit close to the top of that range, right next to E2M1's actual largest value, 6, but flooring cannot tell 6.8 apart from, say, 4.1 — both floor to the same exponent, 2, even though only one of them is close to overflowing.

NVIDIA's version keeps this information instead. Rather than flooring $\text{amax}(H)$ on its own, it first divides $\text{amax}(H)$ by $LPTtype$'s actual largest value: $\text{amax}(H) / \text{amax}(LPTtype) = 6.8/6 = 1.1\bar{3}$. This one number says exactly how far past E2M1's range $\text{amax}(H)$ currently sits: here, about 13% past it. Equation (6) could never say this — the most it could say is which pair of powers of two $\text{amax}(H)$ falls between.

Change 2: round up, not down. Having kept that information, NVIDIA's version then takes $\log_2$ of this ratio and rounds it *up* (ceiling), not down. Whenever $\text{amax}(H)$ is even slightly past $LPTtype$'s range, the ratio is greater than 1, so $\log_2$ of it is greater than 0. Rounding that up, instead of flooring it back down to 0 the way Equation (6) effectively does, is what guarantees the resulting scale factor actually brings $\text{amax}(H)$ back under $LPTtype$'s range — not just into the same pair of powers of two it started in.

Combining both changes gives

$$X = 2^{\text{clamp}(\lceil \log_2(\text{amax}(H) / \text{amax}(LPTtype)) \rceil, -127, 127)} \quad (7)$$

The same example, fixed.

Applied to the same group, $\text{amax}(H) = 6.8$ and $\text{amax}(\mathbf{e2m1}) = 6$, Equation (7) gives

$$\lceil \log_2(6.8/6) \rceil = \lceil \log_2 1.1\overline{3} \rceil = \lceil 0.18 \rceil = 1,$$

so $X = 2^1 = 2$ instead of $X = 1$: an actual scale factor, not a no-op one. Dividing 6.8 by 2 gives 3.4, safely inside **E2M1**'s range, so this group no longer needs to fall back on saturation.

This fix is not free, though. Rounding up guarantees the group's peak value never overflows once divided by X , at the cost of sometimes choosing an X up to twice as large as the tightest possible fit — which pushes some of the group's smaller values closer to **E2M1**'s underflow floor than a perfectly tight scale would. Either way, both formulas clamp the resulting exponent to $[-127, 127]$, simply because that is all **E8M0**'s 8-bit, bias-127 exponent field can hold (Table 13). For this reason — and because Equation (6) can degenerate into a no-op scale, as shown above — the rest of this paper computes block scales with Equation (7), not Equation (6).

9.5 Efficient, Bit-Manipulation-Based Computation of the Scale Factor

Equation (7) looks expensive: a division, a logarithm, and a ceiling. Hardware trying to save cycles cannot afford any of those. In practice, none of them are actually needed — Equation (7) can be computed by reading bits directly out of $\text{amax}(H)$'s own floating-point representation.

The key bit-manipulation trick.

Turning the closed-form formula of Equation (7) into efficient bit-manipulation takes four short steps.

Step 1: write both numbers in mantissa-times-power-of-two form.

$$\text{amax}(H) = M_a \times 2^{e_a}, \quad \text{amax}(LPT\text{Type}) = M_b \times 2^{e_b}, \quad M_a, M_b \in [1, 2) \quad (8)$$

M_a and e_a are read off $\text{amax}(H)$'s bit pattern, interpreted in $HPT\text{Type}$ (the type of H 's values): M_a directly from the mantissa field, e_a from the exponent field minus $HPT\text{Type}$'s own bias, $HPT\text{Bias}$ (Table 3 — 127 for FP32/BF16, 15 for FP16). M_b and e_b come from $\text{amax}(LPT\text{Type})$'s bit pattern the same way (in practice precomputed once, offline, as the constants $LPT\text{MaxMant}$ and $LPT\text{MaxExp}$).

Step 2: take the ratio, then its $\log_2$.

$$\frac{\text{amax}(H)}{\text{amax}(LPT\text{Type})} = \frac{M_a}{M_b} \times 2^{e_a - e_b} \quad (9)$$

$$\log_2 \frac{\text{amax}(H)}{\text{amax}(LPT\text{Type})} = (e_a - e_b) + \log_2 \frac{M_a}{M_b} \quad (10)$$

Step 3: bound the leftover fractional term.

Since $M_a, M_b \in [1, 2)$, their ratio M_a/M_b always lands in $(0.5, 2)$, so $\log_2(M_a/M_b)$ is a fraction strictly between -1 and 1 , in one of three cases:

$$\log_2 \frac{M_a}{M_b} \in \begin{cases} (-1, 0) & M_a < M_b \\ 0 & M_a = M_b \\ (0, 1) & M_a > M_b \end{cases} \quad (11)$$

Step 4: apply the ceiling.

Equation (10) is a whole number — $(e_a - e_b)$ — plus — a fraction. Taking the ceiling of a whole number plus a fraction in each of the three ranges above collapses to just two outcomes:

$$\left\lceil \log_2 \frac{\text{amax}(H)}{\text{amax}(LPT\text{ype})} \right\rceil = (e_a - e_b) + \begin{cases} 1 & \text{if } M_a > M_b \\ 0 & \text{if } M_a \leq M_b \end{cases} \quad (12)$$

Reading Equation (12) off in hardware.

- M_b and e_b are precomputed constants, already sitting there — no fields to read. e_b 's own bias subtraction, the same kind e_a needs next, is already taken care of, exactly as Step 1 described: it happened once, offline, when $LPTMaxExp$ was computed, so nothing about it is left to do here.
- e_a is one subtraction away: read $\text{amax}(H)$'s raw exponent field, interpreted in $HPT\text{ype}$, and subtract $HPT\text{ype}$'s own bias, $HPTBias$, to undo the biasing Section 3.2 described.
- M_a takes one extra piece of care, when it is compared against M_b : both are normal mantissas, normalized to $[1, 2)$, so they share the same implicit leading 1 — only the fractional bits after it can differ, and those bits are what decide $M_a > M_b$ or $M_a \leq M_b$. But $HPT\text{ype}$ and $LPT\text{ype}$ can store that fraction in different bit widths, so the two fraction fields are not directly comparable as stored. The fix is simple: pad $LPT\text{ype}$'s largest-normal mantissa field with zero bits on the low end until it is the same width as $HPT\text{ype}$'s own mantissa field. Once both fractions sit at the same bit width, comparing them is just comparing two plain unsigned integers.
- **Putting it together:** Equation (7) is therefore nothing more than: read $\text{amax}(H)$'s exponent field and subtract $HPTBias$, subtract $LPTMaxExp$, compare two width-aligned mantissa fields, add 0 or 1, and clamp.

Applied once more to the running example, $\text{amax}(H) = 6.8 = 1.7 \times 2^2$ and $\text{amax}(\mathbf{e2m1}) = 6 = 1.5 \times 2^2$: $e_a = e_b = 2$ (already the real, bias-subtracted exponents from Step 1's decomposition above, not raw bit-pattern fields), and $M_a = 1.7 > M_b = 1.5$, so the adjustment is 1, giving $2 - 2 + 1 = 1$ and $X = 2^1 = 2$ — the same answer as before, reached without ever computing a logarithm.

Special values in H .

H 's high-precision values can be normal, sub-normal, zero, $\pm\infty$, or NaN; E8M0 can only represent a power of two or NaN, so the scale computation has to make an explicit decision in four cases. Neither NVIDIA nor the OCP MX specification publishes this level of implementation detail, so what follows is one sound design for each case, not a documented industry standard:

Special Case 1: H contains at least one NaN.

Set X to E8M0's NaN pattern — the whole block's data is suspect. This case matches the published MX semantics directly: “if $X = \text{NaN}$, then $v_i = \text{NaN}$ for all i ” [42].

Special Case 2: H contains at least one $\pm\infty$ (no NaN).

Exclude it from $\text{amax}(H)$ — an infinite scale would push every other, finite element to zero. The excluded infinite element still needs a final down-scaled value: it gets one later, during down-scaling, as per Table 14. This is a deliberate choice, not a documented one — for example, Microsoft's reference MX implementation instead lets an infinite amax poison the *entire* block's

scale to NaN, a more conservative rule than excluding just the infinite element [18].

Special Case 3: H contains only zeros (plus any excluded $\pm\infty$).

All values in H are 0, so any scale reconstructs them exactly; $X = 2^0 = 1$ is simply the default choice.

Special Case 4: H contains only zeros and sub-normals (its peak magnitude is sub-normal).

Steps 1–4 assume $\text{amax}(H)$ is a normal number. That is what makes $M_a \in [1, 2)$ true: a normal mantissa carries an implicit leading 1. A sub-normal mantissa has no such leading 1, so that assumption fails. The fix is to convert $\text{amax}(H)$ into the same normal-number shape first, then apply Steps 1–4 and read the result off in hardware as before.

To do this, shift the mantissa left, one bit at a time, until a 1 bit reaches the position the implicit leading bit would occupy. Let s be the number of shifts performed. Even with zero shifts, a sub-normal value already sits one exponent below the normal range’s floor, $1 - \text{HPTBias}$. Each shift lowers the true exponent by one more. So the resulting exponent is $-\text{HPTBias} - s$.

A worked example.

Take an FP32 value ($\text{HPTBias} = 127$) whose exponent field is all zeros — so it is sub-normal — and whose mantissa field is $00101000000000000000000_2$. Read directly, this is $0.00101_2 \times 2^{1-127} = 1.01_2 \times 2^{-129}$.

Running the shift: the leading bit is 0, so shift ($s = 1$); still 0, so shift again ($s = 2$); now the leading bit is 1, so the loop stops. The exponent is $-\text{HPTBias} - s = -127 - 2 = -129$, matching the direct reading above. One more shift drops that leading 1 (now implicit) and pads a 0 at the low end, leaving a stored mantissa of $01000000000000000000000_2$ — together, $1.01_2 \times 2^{-129}$, exactly the value we started with.

Algorithm 1 puts all of this together.

Algorithm 1: Efficient, bit-manipulation-based computation of a block’s scale factor X

Input:

- block $H = \{h_1, \dots, h_k\}$ of high-precision values, stored in format HPTType
- HPTBias — HPTType ’s own exponent bias
- LPTMaxExp — the target type’s precomputed largest-normal exponent
- LPTMaxMant — the target type’s precomputed largest-normal mantissa field, left-aligned to the same bit width as HPTType ’s own mantissa field

Output:

- X — the shared scale factor for H (or NaN)

Method:

// steps 1–12: compute $\text{amax}(H)$, the block’s largest finite magnitude (return immediately on NaN; skip $\pm\infty$)

- 1: $\text{amax} \leftarrow 0$
- 2: **FOR** $i \leftarrow 1$ **TO** k **DO**
- 3: **IF** h_i is NaN **THEN**

```

4:      return  $X \leftarrow \text{NaN}$     // Above Special Case 1
5:  ENDIF
6:  IF  $h_i = \pm\infty$  THEN
7:      continue    // Above Special Case 2
8:  ENDIF
9:  IF  $|h_i| > \text{amax}$  THEN
10:       $\text{amax} \leftarrow |h_i|$ 
11:  ENDIF
12: ENDFOR

    // steps 13–15: no finite non-zero element was found in H
13: IF  $\text{amax} = 0$  THEN
14:      return  $X \leftarrow 2^0$     // Above Special Case 3
15: ENDIF

    // steps 16–26: decompose amax into exponent e and mantissa m, normalizing first
    // if amax is sub-normal
16:  $(E_{\text{field}}, m) \leftarrow$  the raw exponent field and mantissa field of  $\text{amax}$ 's bit pattern
17:  $e \leftarrow E_{\text{field}} - \text{HPTBias}$ 
18: IF  $E_{\text{field}} = 0$  THEN    // Above Special Case 4
19:       $s \leftarrow 0$ 
20:  WHILE  $m$ 's leading bit  $\neq 1$  DO
21:       $m \leftarrow 2m$ 
22:       $s \leftarrow s + 1$ 
23:  ENDWHILE
24:   $e \leftarrow -\text{HPTBias} - s$ 
25:   $m \leftarrow 2m$ 
26: ENDIF

    // steps 27–31: compare mantissas to get the rounding carry bit c
27: IF  $m > \text{LPTMaxMant}$  THEN
28:       $c \leftarrow 1$ 
29: ELSE
30:       $c \leftarrow 0$ 
31: ENDIF

    // steps 32–33: combine everything into the final scale factor and return

```

```

32:  $X_{exp} \leftarrow \text{clamp}(e - LPTMaxExp + c, -127, 127)$ 
33: return  $X \leftarrow 2^{X_{exp}}$ 

```

9.6 Special-Case Conversion: Saturation vs. Overflow

Table 14 captures how each low-precision type handles special numbers, since *LPT*’s handling of these deviates from *HPT*’s. Two conventions are in use: SAT (saturate to the largest/smallest finite representable value) and OVF (produce an infinity or NaN result, where the target format supports it).

Table 14: Special-value conversion behavior into each low precision type. SAT = result saturates to min_normal/max_normal; OVF = result overflows to $\pm\infty$ or propagates NaN.

Source value	E4M3		E5M2		E2M3	E3M2	E2M1
	SAT	OVF	SAT	OVF	SAT	SAT	SAT
NaN	NaN	NaN	NaN	NaN	N/A	N/A	N/A
$-\infty$	min_normal	NaN	min_normal	$-\infty$	N/A	N/A	N/A
$+\infty$	max_normal	NaN	max_normal	$+\infty$	N/A	N/A	N/A
<min_normal	min_normal	NaN	min_normal	$-\infty$	min_normal	min_normal	min_normal
>max_normal	max_normal	NaN	max_normal	$+\infty$	max_normal	max_normal	max_normal

9.7 Quantization

Quantizing high-precision values $H = \{h_1, \dots, h_k\}$ into low-precision values $L = \{l_1, \dots, l_k\}$ is a three-step process. First, compute the block scale X , following Algorithm 1. Second, down-scale H : divide every element by X , producing $DH = \{dh_1, \dots, dh_k\}$ (Algorithm 2). Third, down-convert DH into L : round every element onto the nearest value *LPT* can actually represent (Algorithm 3). Algorithm 4 carries out all three steps.

Algorithm 2: Down-scaling a block

Input:

- block $H = \{h_1, \dots, h_k\}$
- block scale X

Output:

- $DH = \{dh_1, \dots, dh_k\}$ — down-scaled block

Method:

```

1: IF  $X = \text{NaN}$  THEN
2:   FOR  $i \leftarrow 1$  TO  $k$  DO
3:      $dh_i \leftarrow \text{NaN}$ 
4:   ENDFOR
5: ELSE

```

```

6:   FOR  $i \leftarrow 1$  TO  $k$  DO
7:      $dh_i \leftarrow h_i / X$ 
8:   ENDFOR
9: ENDIF
10: return  $DH$ 

```

Algorithm 3: Down-converting a block

Input:

- $DH = \{dh_1, \dots, dh_k\}$ — down-scaled block
- $LPTtype$ — target low precision type
- $RoundMode$ — how to round a value onto $LPTtype$'s nearest representable value

Output:

- $L = \{l_1, \dots, l_k\}$ — quantized block

Method:

```

1: FOR  $i \leftarrow 1$  TO  $k$  DO
2:   IF  $dh_i \in \{\text{NaN}, +\text{Inf}, -\text{Inf}, LPTtype\text{-out-of-range}\}$  THEN
3:      $l_i \leftarrow \text{SPECIALCONVERSION}(dh_i)$  // As per Table 14
4:   ELSE
5:      $l_i \leftarrow \text{Round}(dh_i, RoundMode)$ 
6:   ENDIF
7: ENDFOR
8: return  $L$ 

```

Algorithm 4: MXFP block quantization

Input:

- block $H = \{h_1, \dots, h_k\}$
- $HPTtype$ — high precision type of values from H
- $LPTtype$ — target low precision type
- $RoundMode$ — how to round a value onto $LPTtype$'s nearest representable value

Output:

- block scale X
- $L = \{l_1, \dots, l_k\}$ — quantized block

Method:

```

1:  $HPTBias \leftarrow HPTtype$ 's own exponent bias
2:  $LPTMaxExp \leftarrow LPTtype$ 's precomputed largest-normal exponent

```

```

3:  $LPTMaxMant \leftarrow LPTType$ 's precomputed largest-normal mantissa field, left-aligned
   to  $HPTType$ 's own mantissa field
4:  $X \leftarrow \text{Algorithm\_1}(H, HPTBias, LPTMaxExp, LPTMaxMant)$  // Compute scale factor
5:  $DH \leftarrow \text{Algorithm\_2}(H, X)$  // Down-scale  $H$ 
6:  $L \leftarrow \text{Algorithm\_3}(DH, LPTType, RoundMode)$  // Down-convert  $DH$ 
7: return  $X, L$ 

```

Of these four algorithms, only Algorithm 3 — down-converting a block — is typically implemented directly in hardware: rounding a value onto a narrower format's nearest representable value is a routine format-conversion primitive that general-purpose float units already support. Algorithms 1, 2 and 4 are, by contrast, typically software-level computations: computing a block's scale factor requires a reduction across the whole block, and orchestrating the full pipeline is just control flow — neither of which the matrix-multiply hardware performs natively.

9.8 Dequantization

Conceptually, dequantizing L back into high-precision values is a two-step process — the mirror image of quantization. First, up-convert L into high-precision values, producing $DH = \{dh_1, \dots, dh_k\}$ (Algorithm 5). Second, up-scale DH : multiply every element by the block's scale factor X , producing $H = \{h_1, \dots, h_k\}$ (Algorithm 6). Algorithm 7 carries out both steps.

Algorithm 5: Up-converting a block

Input:

- $L = \{l_1, \dots, l_k\}$ — quantized block
- $HPTType$ — target high precision type

Output:

- $DH = \{dh_1, \dots, dh_k\}$ — up-converted block

Method:

```

1: FOR  $i \leftarrow 1$  TO  $k$  DO
2:    $dh_i \leftarrow l_i$  re-encoded in  $HPTType$  // Exact: no rounding needed
3: ENDFOR
4: return  $DH$ 

```

Algorithm 6: Up-scaling a block

Input:

- $DH = \{dh_1, \dots, dh_k\}$ — up-converted block
- block scale X

Output:

- $H = \{h_1, \dots, h_k\}$ — up-scaled block

Method:

```

1: IF  $X = \text{NaN}$  THEN
2:   FOR  $i \leftarrow 1$  TO  $k$  DO
3:      $h_i \leftarrow \text{NaN}$ 
4:   ENDFOR
5: ELSE
6:   FOR  $i \leftarrow 1$  TO  $k$  DO
7:     IF  $dh_i \in \{\text{NaN}, +\text{Inf}, -\text{Inf}\}$  THEN
8:        $h_i \leftarrow dh_i$     // Passed through unscaled
9:     ELSE
10:       $h_i \leftarrow X \cdot dh_i$ 
11:    ENDIF
12:  ENDFOR
13: ENDIF
14: return  $H$ 

```

Algorithm 7: MXFP block dequantization

Input:

- $L = \{l_1, \dots, l_k\}$ — quantized block
- $HPT\text{Type}$ — target high precision type
- block scale X

Output:

- $H = \{h_1, \dots, h_k\}$ — dequantized block

Method:

```

1:  $DH \leftarrow \text{Algorithm\_5}(L, HPT\text{Type})$     // Up-convert  $L$ 
2:  $H \leftarrow \text{Algorithm\_6}(DH, X)$     // Up-scale  $DH$ 
3: return  $H$ 

```

Of the three dequantization algorithms, Algorithm 5 — up-converting a block — is almost always inherent to the hardware itself: accumulating a low-precision product into a high-precision accumulator, which every matrix-multiply unit already does, is exactly what up-converting means. Algorithm 6 — up-scaling a block — is more situational: whether it happens in hardware or in software depends on the operation being performed and how its pipeline is built. The common case is a dot product of two low-precision vectors, illustrated next.

In the case of a dot product of two low-precision vectors, *dequantization* is typically fused into the tensor operation itself and performed by the matrix-multiply hardware, rather than materialized as a separate step. Consider two MX-compliant tensor blocks:

- $A = \{X_A, [L_i^A]_{i=1}^k\}$
- $B = \{X_B, [L_i^B]_{i=1}^k\}$

Their dot product is defined as

$$C = \text{Dot}(A, B) = X_A \cdot X_B \cdot \sum_{i=1}^k L_i^A \cdot L_i^B \quad (13)$$

where the hardware performs the low-precision multiply-accumulate $\sum_i L_i^A L_i^B$ directly in the narrow format’s native arithmetic, keeps the accumulated result in a high-precision format, and then applies the two scale factors X_A, X_B only once, to that accumulated result — rescaling the entire block’s contribution to the output in a single step, rather than rescaling every individual product.

10 NVIDIA NVFP4: A Refined 4-bit Format

Applying the strict MXFP recipe of Section 9 to FP4 gives MXFP4: 32-wide blocks of E2M1 values, each block sharing one power-of-two E8M0 scale. In practice, NVIDIA found MXFP4 carries a real accuracy risk relative to FP8 [24]: in pretraining experiments, MXFP4 needed roughly 36% more training tokens than NVFP4 to reach the same loss [30]. NVIDIA responded with a modified 4-bit quantization scheme, **NVFP4**, that makes three changes to strict MXFP4 at once: a different scale type, a smaller block size, and a second, coarser scaling level [24, 30].

10.1 Two-Level Scaling: Tensor and Block

1. **Block-level scaling.** NVFP4 changes two things about the per-block scale compared to strict MXFP4. First, the scale type: MXFP’s power-of-two, 8-bit E8M0 scale is replaced with an *unsigned* 4-exponent/3-mantissa FP8 scale, **UE4M3** (the “U” denoting that the sign bit is fixed at zero, restricting the type to the positive half of ordinary E4M3’s range). Because E4M3 carries a 3-bit mantissa, the block scale is no longer restricted to a power of two: it can take on *fractional* values, which lets the scale track the block’s true peak magnitude far more closely than a power-of-two scale can. Second, the block size: NVFP4 halves the MXFP scaling-block size from 32 down to 16 values per shared scale, at the cost of roughly doubled scale-storage overhead (one UE4M3 scale per 16 elements rather than per 32), in exchange for scale factors that track local magnitude variation more tightly.
2. **Tensor-level scaling.** This is the same block-scale idea, just with the block boundary pulled all the way out: instead of a 16-wide block, the block is now the *entire* tensor — every element in the tensor shares this one scale. And unlike the per-block scale, which is rounded down into the narrow UE4M3 type, this tensor-wide scale is kept as an ordinary **float32** value, with no rounding at all. NVFP4 uses both scales together — the fine-grained, 16-wide block scale for local magnitude variation, and this coarse-grained, tensor-wide **float32** scale on top of it — and it is this combination that lets NVFP4 recover the accuracy that strict, single-level MXFP4 block scaling loses.

10.2 Why Two-Level Scaling?

Section 10.1 named the two levels; it is worth being explicit about why NVFP4 needs both, rather than simply swapping E8M0 for UE4M3 and stopping there.

E8M0 and UE4M3 trade off range against precision in opposite directions. E8M0 is an 8-bit pure exponent, so it can represent scale factors across an enormous range (2^{-127} to 2^{127}), but only at power-of-two granularity: if a block’s true ideal scale — the exact ratio $\text{amax}(H)/\text{amax}(\text{e2m1})$ needed to bring a block’s peak magnitude down to E2M1’s range — falls between two powers of two, E8M0 must round up to the next power of two — never to nearest, since Section 9.4 already established that rounding down risks overflow — and that rounding error eats directly

into FP4’s already tiny 4-bit budget — the precision problem Section 10.1 described. UE4M3 fixes that precision problem: with a 3-bit mantissa, it can store a fractional value close to that same ideal scale, instead of being forced to jump all the way up to the next power of two. But it pays for that precision with range: a 4-exponent-bit format tops out at 448, nowhere near E8M0’s reach.

That trade-off creates a new problem the block scale alone cannot solve. The number the block scale actually has to store is $\text{amax}(H)/\text{amax}(\text{e2m1})$ — how many times larger a block’s peak magnitude is than E2M1 can represent. A tensor’s raw magnitudes vary hugely across layers and models, so this ratio can easily fall well outside $[0, 448]$, forcing UE4M3 to saturate — the same kind of failure NVFP4 was trying to escape in the first place, just triggered by range instead of granularity.

The tensor-wide `float32` scale exists to head this off. It pre-normalizes the entire tensor *before* any block scale is computed, so that every block’s local scale ratio is guaranteed to land inside UE4M3’s representable range once it is finally rounded [22, 30]. `float32` is used for this step specifically because it has to absorb the tensor’s full, uncompressed dynamic range, whatever that happens to be. Put simply: the block scale’s job is to measure each block precisely; the tensor scale’s job is to first bring every block within reach of that precise measurement.

10.3 Scaling Factor Computation

Turning the two scaling levels of Section 10.1 into an actual number to quantize each element takes three steps [22, 29]: (1) compute the tensor scale, (2) compute the block scale, (3) then fold both into a single per-block conversion factor.

Step 1: the tensor scale.

Let $\text{amax}(\text{e2m1}) = 6$ be the largest finite value representable in E2M1 (Table 11), and $\text{amax}(\text{e4m3}) = 448$ the largest finite value representable in E4M3 (Table 8). The tensor-level scale exists to bring the whole tensor’s dynamic range within reach of UE4M3, the block scale’s own storage type. Given $\text{amax}(T)$ — the peak magnitude across the entire tensor T being quantized — the `float32` tensor-wide scale is

$$S_T^{f32} = \frac{\text{amax}(\text{e2m1}) \cdot \text{amax}(\text{e4m3})}{\text{amax}(T)} \quad (14)$$

Step 2: the block scale.

The block-level scale then handles what the tensor scale alone cannot: local magnitude variation within the tensor. For a block $H = \{h_1, \dots, h_{16}\}$ with peak magnitude $\text{amax}(H)$, the block scale is first computed in `float32`, the same way the tensor scale was,

$$S_B^{f32} = \frac{\text{amax}(H)}{\text{amax}(\text{e2m1})} \quad (15)$$

and then combined multiplicatively with the tensor scale before being rounded into the UE4M3 block-scale type:

$$S_B^{e4m3} = \text{e4m3}(S_T^{f32} \cdot S_B^{f32}) \quad (16)$$

Folding the tensor scale in *before* rounding, rather than after, is what makes S_B^{e4m3} account for the block’s local magnitude *together with* where the tensor scale has already placed it, rather than the block’s raw magnitude alone.

This also proves, rather than just asserts, the guarantee Section 10.2 described in words. Multiplying Equations (14) and (15) together — exactly the quantity Equation (16) rounds into

UE4M3 — gives

$$S_T^{f32} \cdot S_B^{f32} = \frac{\text{amax}(\mathbf{e2m1}) \cdot \text{amax}(\mathbf{e4m3})}{\text{amax}(T)} \cdot \frac{\text{amax}(H)}{\text{amax}(\mathbf{e2m1})} = \text{amax}(\mathbf{e4m3}) \cdot \frac{\text{amax}(H)}{\text{amax}(T)},$$

since $\text{amax}(\mathbf{e2m1})$ cancels. Because H is a 16-element block drawn from T , $\text{amax}(H) \leq \text{amax}(T)$ always, so $\text{amax}(H)/\text{amax}(T)$ lies in $[0, 1]$, and therefore

$$S_T^{f32} \cdot S_B^{f32} \in [0, \text{amax}(\mathbf{e4m3})] = [0, 448],$$

UE4M3’s exact representable range — whatever the tensor’s raw magnitude turns out to be. Equation (16) can therefore never be asked to round a value larger than UE4M3 can hold.

Step 3: the block conversion factor.

Quantizing an element still needs one single scale to operate it by, not two separate scales. The tensor scale and the rounded block scale are combined into that single per-block `float32` conversion factor,

$$R_B^{f32} = \frac{S_T^{f32}}{\mathbf{f32}(S_B^{e4m3})} \quad (17)$$

which Algorithm 8 applies directly to every element of the block.

What does this number actually amount to? Suppose, for a moment, that Step 2’s rounding into UE4M3 introduced no error at all, so that $S_B^{e4m3} = S_T^{f32} \cdot S_B^{f32}$ exactly. Equation (17) would then collapse to

$$R_B^{f32} = \frac{S_T^{f32}}{S_T^{f32} \cdot S_B^{f32}} = \frac{1}{S_B^{f32}} = \frac{\text{amax}(\mathbf{e2m1})}{\text{amax}(H)},$$

using Equation (15). Strict, single-level block quantization (Section 9.7) quantizes by *dividing* each element by a scale X , ideally $X = \text{amax}(H)/\text{amax}(\mathbf{e2m1})$ before E8M0 forces it to round up to a power of two (Equation (7)). Dividing by X and multiplying by $1/X$ are the same operation, so R_B^{f32} here is exactly that $1/X$: NVFP4’s multiplicative Algorithm 8 and strict MXFP4’s divisive down-scaling step apply the identical scale, just expressed in two different but equivalent ways. In general S_B^{e4m3} is not exact — UE4M3 still rounds it, just at fractional-mantissa precision — so R_B^{f32} only approximates $\text{amax}(\mathbf{e2m1})/\text{amax}(H)$, off by whatever small rounding error UE4M3 introduced in Step 2, rather than by the much coarser power-of-two rounding error E8M0 would introduce directly. Step 3 is where the two-level scale built up in Steps 1 and 2 pays off: it hands each element almost exactly the ratio a perfectly precise single-level scale would have used, which is what Section 10.2 argued UE4M3 makes possible and E8M0 does not.

10.4 NVFP4 Quantization

Given the block conversion factor R_B^{f32} , each element of the block is quantized directly into E2M1, as shown in Algorithm 8.

Algorithm 8: NVFP4 block quantization

Input:

- block $H = \{h_1, \dots, h_{16}\}$
- block conversion factor R_B^{f32}

Output:

- $L = \{l_1, \dots, l_{16}\}$ — quantized block in E2M1

Method:

- 1: **FOR** $i \leftarrow 1$ **TO** 16 **DO**
- 2: $l_i \leftarrow \text{e2m1}(\text{f32}(h_i) \cdot R_B^{f32})$
- 3: **ENDFOR**

11 Hardware Realization of Low-Precision Types

11.1 Hardware Challenges of Realizing Low-Precision Types

A general-purpose instruction set architecture (ISA) typically exposes only a small, fixed set of built-in scalar types — integers at a few widths, and `float32/float64` (increasingly `float16` too) — each backed by dedicated execution hardware (an integer ALU, a floating-point unit) and each individually addressable at the hardware’s natural memory granularity, the byte. Two properties of that arrangement quietly do a lot of work:

1. the opcode alone is normally enough to say which type is meant (an integer `ADD` and a floating-point `FADD` never need to be told apart any other way); and
2. every value, however narrow, still occupies at least one whole byte in memory, so it can be addressed, loaded, and stored like anything else.

The low-precision formats surveyed in this paper break both of these assumptions at once.

The opcode-combinatorics problem.

In principle, nothing stops an ISA from giving E4M3 and E5M2 their own distinct opcodes, the same way `ADD` and `FADD` are distinct opcodes regardless of shared bit width. What rules this out in practice is that a block-scaled matrix-multiply instruction does not select just one type: it independently selects a format for operand A , a (possibly different) format for operand B , and a type and block size for the shared scale factor [44] — drawn from a set of seven distinct narrow-precision types [25]. Giving every legal combination of these its own hardcoded opcode would multiply the opcode count rather than add to it, and instruction opcode space is a small, fixed-width field — not something that can be spent on a combinatorial product of one instruction’s independent operand choices.

The sub-byte addressability problem.

E2M1 (FP4, 4 bits) and both FP6 variants (6 bits) are narrower than a byte, but essentially every practical memory system — DRAM, caches, the load/store path — addresses memory at byte granularity as its smallest unit. E2M1 divides a byte evenly (two values per byte); the two FP6 variants do not divide a byte at all: only a group of four 6-bit values (24 bits) lands back on a whole-byte boundary. Neither format can be given its own hardware address the way an `int32` or `float32` can.

11.2 Mitigation: Packed Storage and Qualifier-Encoded Types

Hardware that supports these formats resolves both problems with the same underlying move: separate *how bits are stored* from *how those bits are to be interpreted*.

Packed storage.

At rest, low-precision values are kept as dense, generically-packed bytes — two E2M1 values to a byte, four FP6 values to three bytes — with no per-value type tag and no per-value byte address. Only the packed group, not a single element inside it, can be addressed on its own.

Qualifier-encoded type.

The actual format identity — which specific encoding is meant for each operand, and at what block/group size — is supplied separately, as metadata carried by the instruction (an operand descriptor or instruction qualifier) rather than baked into the opcode itself. This is what turns Section 11.1’s multiplicative opcode-combinatorics problem into an additive one: instead of one opcode per legal combination of *A*-type, *B*-type, and scale type/size, the instruction carries one small field per independent choice. It also sharpens, rather than replaces, a convention already present in classic ISAs: a register or memory location never really “knows” whether it holds an integer or a float — an `ADD` and an `FADD` interpret the identical bit pattern differently — low-precision hardware just makes that same type-is-instruction-metadata idea explicit and multi-dimensional, because now several independent choices need to be carried, not just one.

Together, these two moves are what let hardware support an open-ended, growing family of narrow formats without redesigning memory addressing every time a new one appears: the storage layer only ever needs to move and pack raw bytes, and the type-specific logic lives entirely at the instruction/qualifier layer, where it is comparatively cheap to extend.

11.3 NVIDIA Blackwell: A Concrete Illustration

NVIDIA’s Blackwell architecture — the same architecture NVFP4 (Section 10) was designed for — implements exactly this two-part scheme, and its own specification documents the mechanism in enough detail to serve as a concrete illustration.

Packed storage, worked through.

The OCP Microscaling specification that Section 9’s formats build on deliberately leaves memory layout unspecified: “the layout of the block in physical memory is not prescribed in this specification” [31], so packing is left entirely to the hardware vendor. In NVIDIA’s implementation, tensors are kept fully packed in global memory (GMEM) — two E2M1 values per byte, four FP6 values per three bytes — with no wasted bits, preserving the bandwidth and footprint savings that motivated using a narrow format in the first place [2]. Before a Blackwell tensor core can consume this data, however, it must sit in shared memory (SMEM) in a specific 16-byte-aligned layout, so a hardware unit dedicated to moving data between GMEM and SMEM, the Tensor Memory Accelerator (TMA) — capable of transfers in either direction, though only the GMEM-to-SMEM load direction matters here — expands each packed group of 16 elements into that aligned form during the transfer itself: its `.b4x16_p64` load variant takes 16 packed 4-bit elements (8 bytes, dense) and appends 8 bytes of padding, and its `.b6x16_p32` variant takes 16 packed 6-bit elements (12 bytes, dense) and appends 4 bytes of padding — both landing on exactly 16 bytes [2]. The expansion happens once, in dedicated hardware, during a single memory transfer; nothing is ever stored in the wasteful, padded form at full memory bandwidth.

Qualifier-encoded type, worked through.

At the true ISA level, the PTX instruction that consumes this data is `tcgen05.mma`, and it carries the type information as an explicit qualifier on the instruction itself rather than as a separate opcode: `.kind::mx8f6f4`. The name says exactly what it covers — “mx” for microscaled (block-scaled), “f8f6f4” for the FP8, FP6, and FP4 families together — so this one qualifier tells the hardware only which broad family of block-scaled narrow-precision operation is meant, not

yet which specific encoding within it. The specific encoding for each operand, and the type and size of the scale factor, are what get carried separately, exactly as Section 11.2 described: for NVFP4, that separate information says operand *A* and operand *B* are both E2M1, the scale factor is UE4M3, and the block size is 16; for MXFP4 it would instead say E2M1 operands with an E8M0 scale at block size 32. The PTX specification documents a dedicated packing-format section for exactly this qualifier [27]. In summary: PTX does not expose low-precision types as first-class types at all — it exposes one generic, family-level opcode qualifier, and encodes everything specific about which type is actually meant as separate metadata alongside it.

A detail worth flagging, not claiming.

The TMA’s packing group size is 16 elements for both the 4-bit and 6-bit load variants above, and 16 is also NVFP4’s own block-scaling group size (Section 10.1). Whether this is deliberate co-design — sizing the memory-transfer granularity to match the scaling granularity — or simply a convenient shared alignment unit is not something the sources consulted here confirm either way; it is noted as an observation, not asserted as a fact.

12 Framework-Level Realization of Low-Precision Types

Section 11 looked at the very bottom of the AI stack: how low-precision types are actually stored and computed on, once the data reaches the hardware. This section moves to the very top of the same stack and looks at low-precision types from the other end: how AI frameworks such as PyTorch and JAX — the tools a machine learning engineer actually writes code in — represent these types, and the choices each framework makes to fit them into a tensor system built around a small, fixed set of dtypes.

12.1 Framework Challenges of Representing Low-Precision Types

A general-purpose deep learning framework’s tensor system, like a general-purpose ISA, normally works with only a small, fixed set of built-in scalar dtypes — `float32`, `float64`, `float16`, `bfloat16`, and a handful of integer widths. Two properties of that arrangement quietly do a lot of work, and they are the same two properties Section 11.1 pointed to at the hardware level:

1. the framework’s operator library of various operations — addition, multiplication, matrix multiplication, and so on — is written once, generically, so it works the same way across every built-in dtype, if required by applying the framework’s own type-promotion rules; and
2. every value, however narrow, still occupies at least one whole byte of array storage, so ordinary indexing, slicing, and reshaping work the same way no matter which dtype is involved.

The low-precision formats surveyed in this paper break both of these assumptions again, now at the very top of the stack.

The generic-arithmetic problem.

A raw E4M3 or E2M1 bit pattern is not, on its own, a meaningful number the way a `float32` bit pattern is: as Sections 8 to 10 established, these formats are only usable at all together with a separate scale factor. Because of this, every operation a framework performs on a low-precision type would also need a matching scale that the dtype itself does not carry. Both PyTorch and JAX respond by not extending the generic operator library at all: PyTorch exposes one dedicated function, `torch._scaled_mm`, that takes the scale as an explicit extra argument [40], and JAX relies on the XLA compiler to recognize a cast-scale-dot sequence and fuse it into one

FP8 matrix multiply [26]. This is the same underlying limitation Section 11.1 already found at the hardware level: a fixed, general-purpose operation set cannot be extended, one at a time, to cover every combination of low-precision type and scale.

The sub-byte addressability problem.

E2M1 (FP4) and the FP6 pair are narrower than a byte, but the array libraries both frameworks are built on — PyTorch’s own tensor storage, and JAX’s use of NumPy’s dtype system through the `ml_dtypes` library — address memory at byte granularity, just like the hardware memory systems Section 11.1 described. `ml_dtypes` states the constraint outright: “NumPy does not support types smaller than a single byte,” so its sub-byte float types are stored unpacked, padded up to a byte per element [9]. PyTorch’s core developers hit the same wall when deciding whether to add native FP6 dtypes, and deferred the proposal, citing “unclear packing semantics” [37]: FP6’s four-values-per-three-bytes structure (Section 7) does not reduce to a clean per-byte packing the way FP4’s does.

12.2 Mitigation: Dtype-Level Packing and Object-Level Scale Metadata

Section 11.2 found hardware converging on one shared scheme for both problems at once: pack raw bits densely, and carry the type and scale identity as separate metadata alongside them. At the framework level, the two major frameworks converge on the same overall idea, but split the work between the dtype itself and the metadata layered on top of it in two different ways.

PyTorch: pack the dtype itself.

PyTorch’s FP4 dtype, `torch.float4_e2m1fn_x2`, bakes packing directly into its definition: the trailing `_x2` means every element of this dtype already holds two packed E2M1 values, and PyTorch core provides no separate, unpacked, one-value-per-byte FP4 dtype at all [36, 39]. Because packing is now part of the dtype’s own definition rather than a separate concern, an ordinary operation like viewing or transposing a packed FP4 tensor operates at byte granularity, and it is left to the user to decide whether that byte-level view still means what they intended [36]. Nothing equivalent exists for FP6 in PyTorch core, for exactly the reason given above: its four-values-per-three-bytes grouping does not reduce to a clean per-byte packing the way FP4’s two-values-per-byte grouping does.

The type-and-scale-identity side of the split — the role an instruction qualifier played in Section 11.2 — is filled one layer up, by an ordinary Python object. `torchao`’s `NVFP4Tensor`, for instance, is a tensor subclass whose fields are exactly the packed-storage-plus-metadata split made explicit: a packed `qdata` buffer (two E2M1 values per byte, stored as `torch.uint8`), a block-wise `scale` tensor stored in PyTorch’s `float8_e4m3fn` dtype (used here only for the non-negative UE4M3 values Section 10 already introduced), an optional tensor-wide `per_tensor_scale`, a fixed `block_size` of 16 — matching the block size already established in Section 10.1 — and the tensor’s original high-precision dtype [38]. None of this is hardware or ISA metadata; it is ordinary Python object state, read by `torchao`’s own code before the data ever reaches a kernel.

JAX: leave the dtype unpacked.

`ml_dtypes`, and therefore JAX, takes the opposite position: none of `float8_e4m3fn`, `float8_e5m2`, `float4_e2m1fn`, `float6_e2m3fn`, or `float6_e3m2fn` are packed at the dtype level. Every element takes a full byte of array storage, with the unused high bits simply ignored, regardless of the format’s true bit width [9]. This keeps ordinary array operations — indexing, slicing, reshaping — working exactly the same way they do for any other dtype, at the cost of using two to four times more memory than the value’s true bit width while the array sits at this layer. Dense, hardware-ready packing is deferred entirely to whatever consumes the array afterward, rather than being a property of the array itself.

13 Compiler-Level Realization of Low-Precision Types

Sections 11 and 12 covered the two ends of the AI stack: Section 11 looked at the very bottom — how low-precision types are realized in hardware, and Section 12 looked at the very top — how AI frameworks such as PyTorch and JAX represent these types for a machine learning engineer to write code against. This section covers the ground in between: the compiler stack that turns framework-level (low-precision) computation into the hardware instructions Section 11 described.

Stripped of its many real-world complications, that compiler stack has a simple shape: computation flows through it in one direction, from the framework down to the hardware. A *graph compiler* takes code from the framework in the form of a directed graph, a general-purpose *LLVM compiler* generates the final machine instructions to be executed on the hardware, and *MLIR*, the Multi-Level Intermediate Representation, sits in between both and is what actually connects the two, as sketched in Figure 12.

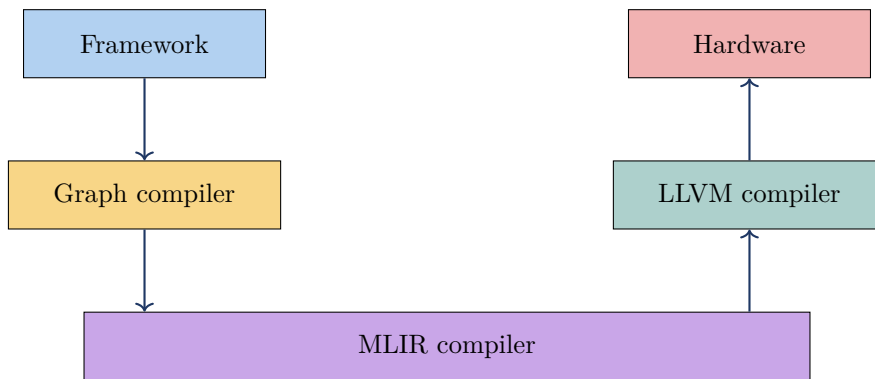


Figure 12: The AI compiler stack in simplified form: computation flows down the stack, from the framework at the top, through a graph compiler, into MLIR, through a general-purpose LLVM compiler, and finally to the hardware at the bottom.

This picture only applies when the framework actually builds and compiles a fresh kernel for that computation, on the fly, right before running it — what PyTorch calls *compile mode*. In practice this decision is made per node, not all at once: once the graph compiler has optimized and fused the computation into its final graph, it can check, for a given node, whether a suitable pre-built kernel is already available in a kernel library; if one is, the framework simply calls that existing kernel, and only the nodes without a matching pre-built kernel are compiled fresh through the rest of the compiler stack depicted in Figure 12 [34].

13.1 Why the Two Ends of the Compiler Stack Stay Generic

Figure 12 places a graph compiler at the framework end and a general-purpose LLVM compiler at the hardware end, with MLIR connecting them underneath. A natural question is which of these three compiler layers actually carries out the lowering of low-precision computation — implementing the quantization and dequantization work Sections 9 and 10 described. The two ends, it turns out, both decline the job.

The graph compiler stays generic.

At the graph level, low-precision computation is mostly kept in the framework’s own representation, with no special handling. XLA’s own FP8 design discussion says this outright: FP8 is added as just two new primitive dtype values, with scaling expressed using the multiply and divide operations HLO already has, not a dedicated FP8-aware graph operation [33]. PyTorch’s native-FP8 proposal follows the same pattern: the dtype fits inside PyTorch’s existing type-promotion

system, and `torch._scaled_mm` (Section 12.1) is traced through as an ordinary opaque call, not rewritten with any low-precision-specific graph logic [35]. In other words, at the graph IR level a low-precision type gets the same treatment it already gets at the framework level (Section 12.1): the generic operator library is left alone, and scaling stays ordinary arithmetic rather than becoming a dedicated construct.

The LLVM backend stays generic too.

The same is true at the LLVM IR level, for the same reason Section 11.1 already gave for hardware: low-precision types are not first-class citizens of the instruction set. LLVM’s own APFloat library does define dedicated software-arithmetic semantics for a wide range of these formats — both FP8 variants, both FP6 variants, and FP4 among them — but none of this is exposed as a first-class LLVM IR type: the enumeration that lists every type an IR value can actually have (`half`, `bfloat`, `float`, and so on) has no entry for any of them [13, 15]. A packed low-precision value reaching an NVPTX intrinsic, for instance, is typed at the IR level as a plain 16-bit integer, and it is the intrinsic being called — not the value’s IR type — that says which specific format those bits mean [11]. In other words, at the LLVM level a low-precision type gets the same treatment it already gets at the true hardware ISA level (Section 11.3).

13.2 Mitigation: MLIR as Low-Precision Lowering Engine

With both ends of the compiler stack declining the job — (a) the graph compiler by keeping low-precision computation expressed in the framework’s own generic operations rather than a dedicated construct, and (b) LLVM by exposing these types only through intrinsics, with plain integers standing in for them at the IR level — the work of turning a block-scaled, packed, low-precision computation into a concrete sequence of instructions has to happen somewhere in between. In practice, it lands on MLIR.

Landing there is not automatic, though. It only works if MLIR’s own infrastructure actually has five specific qualities, worked through here one at a time rather than assumed:

1. representing low-precision types as first-class citizens of its own type system, not merely as raw bytes;
2. importing a packed low-precision tensor from the framework (PyTorch, say) into that first-class representation, sub-byte formats included;
3. having the infrastructure/logic needed to carry out the block-scaling quantization and dequantization work Sections 9 and 10 described;
4. supplying the randomness that stochastic rounding needs during quantization, through an efficient pseudo-random-number-generation implementation; and
5. re-encoding the first-class low-precision type representation into the form LLVM IR/the target ISA actually expects.

A first-class low-precision type system.

This is the most basic requirement, and MLIR already meets it today. Its builtin dialect ships dedicated types for essentially the whole family this paper covers — both FP8 variants and their FNUZ counterparts, both FP6 variants, FP4, and even a dedicated type for the E8M0 scale factor itself [19]. This is a sharp contrast with what Section 13.1 found one level down, at the LLVM IR level, where the type enumeration has no entry for any of these formats at all: MLIR clears this bar, and LLVM does not.

Importing sub-byte tensors without losing their type.

The second requirement sits one layer above MLIR itself: whatever imports a computation graph from the framework into MLIR has to actually use that first-class type as the tensor’s element type, rather than falling back to some generic packed-integer placeholder, even for formats narrower than a byte. This is possible in principle, because of how MLIR’s own tensor abstraction works: a tensor value has no memory address of its own until it is later converted into an addressable buffer, so the sub-byte addressing problem Section 11.1 described for hardware simply does not arise at this stage: a tensor of FP4 elements is just an ordinary, legal type. That problem only reappears once the tensor is lowered toward an actual buffer, and MLIR has a dedicated pass for exactly that later step, packing narrow elements into byte-addressable storage [14]. What this paper does not verify is whether every framework’s own importer takes advantage of this and actually emits the first-class type end to end; that is a per-project detail outside MLIR’s own territory.

The dialects and passes to do the actual lowering.

This third requirement is the heart of the job: once a tensor is properly typed, MLIR needs the dialects and passes to carry out the block-scaling quantization and dequantization work Sections 9 and 10 described — computing a block’s scale factor, quantizing each element against it, and reversing the process on the way back out. MLIR’s general-purpose infrastructure for this — its dialect system, its pattern-rewrite and pass framework, the typed values the first requirement supplies — is built to be extended this way, and a dialect such as `quant` shows the general pattern already exists in-tree [21]. A concrete, ready-made pass that performs the specific OCP Microscaling or NVFP4 algorithm end to end, however, does not appear to live in mainline MLIR itself; that work instead shows up downstream, in vendor- and framework-specific dialects such as Triton’s own MLIR-based dialect [44]. In short, MLIR supplies the extensibility; the concrete algorithm is, today, someone else’s job to plug in — largely because that algorithm’s actual implementation depends on how the target hardware represents and processes low-precision computation in the first place, which is exactly the kind of vendor-specific detail MLIR’s generic layer is not meant to hardcode.

Supplying randomness for stochastic rounding.

The fourth requirement is easy to overlook. Quantizing to a very narrow format can round every value the same direction so consistently that the rounding errors stop cancelling out and instead accumulate as a systematic bias — a real concern during training, where the same small values get quantized over and over. *Stochastic rounding* rounds up or down at random, weighted by how close the true value sits to each side, so the errors cancel out on average instead of compounding. That randomness has to come from somewhere, and here MLIR is thin on the ground: no dialect in its current, upstream list is dedicated to random number generation at all, a gap already flagged in an LLVM discussion years ago that never converged into an accepted dialect [10, 20]. What upstream support does exist is narrow and very recent: MLIR’s own `nvvm` dialect — NVIDIA-target, but still part of the MLIR/LLVM codebase — added a stochastic-rounding mode to its half-precision conversion operations only in late 2025, and even that operation only *consumes* an already-generated random value passed to it as an explicit operand; it does not generate one itself [12]. The actual generation of that randomness is left entirely to the vendor-specific layer below MLIR: a real AMD kernel for stochastic-rounded MXFP4 quantization, for instance, generates the randomness in software with a counter-based algorithm called Philox — reproducible because it is seeded and offset explicitly rather than relying on hidden state — and then feeds those bits to a dedicated hardware instruction that performs the rounded conversion [41].

Re-encoding for the handoff to LLVM.

The fifth requirement mirrors the first: whatever first-class representation MLIR used internally has to be converted back down into whatever LLVM IR actually accepts, since LLVM IR itself has no first-class type for any of these formats. Only the last, mechanical step of this hand-off — turning MLIR’s own `llvm` dialect into actual LLVM IR text — is generic and shared across every target. The re-encoding decision itself is not: working out that, say, an FP8 value should become a plain 16-bit integer paired with a particular NVPTX intrinsic [11] is logic written once for NVIDIA’s target and again, separately, for every other vendor’s target — again largely vendor-specific, based, as with the third requirement, on how the target hardware represents and processes low-precision computation in the first place.

14 Standardization and Outlook

14.1 IEEE P3109: Standardizing 8-bit Floating Point

All of the FP8/FP6/FP4 formats surveyed in Sections 6, 7, 9 and 10 are, at the time of writing, defined by vendor whitepapers and the OCP’s MX specification rather than by a formal IEEE standard. The IEEE P3109 working group, active under the IEEE Standards Association, is working to close that gap specifically for 8-bit binary floating point, with a scope explicitly framed around ML-hardware needs (dynamic range under quantization, hardware cost) rather than the general-scientific-computing goals that shaped IEEE 754 itself [8]. Two FP8 variants — E4M3 and E5M2, matching the encodings already in Section 6.1 — form the starting point of the proposal; contributing organizations have included Arm, Intel, and NVIDIA (via a joint FP8 whitepaper that predates the working group) and Graphcore, AMD, and Qualcomm (via a separate detailed FP8 proposal submitted to the same effort) [43]. Formal machine verification of draft P3109 semantics has also begun, an indication that the effort is maturing toward a citable, implementation-checkable standard rather than remaining purely descriptive [1]. As of this writing, P3109 has not yet been formally ratified; industry practice for FP8 (and, *a fortiori*, for FP6/FP4 and the MX/NVFP4 scaling schemes built on top of them) continues to be set by vendor convention and the OCP specification in the meantime.

15 Conclusion

This paper set out to tell one continuous story: how a computer represents a real number at all, and how that representation has been reshaped, over the last decade, by the specific arithmetic demands of deep learning hardware. Starting from the real number line and a hypothetical infinite-precision representation (Section 2), we built up IEEE 754 single precision field by field (Section 3), and then traced the same sign/exponent/mantissa skeleton as it was reshaped, repeatedly, to serve AI workloads: widened exponent, narrowed mantissa for `bfloat16` and TF32 (Section 5); shrunk to 8 bits, with infinity itself sacrificed for range in E4M3 (Section 6); shrunk further still to 6 and 4 bits, where even NaN and infinity disappear entirely (Section 7); and, finally, made numerically usable again at 4 bits only through the block- and tensor-scaling machinery of MXFP4 and NVFP4 (Sections 9 and 10), with Section 11 then connecting all of this to its hardware realization, Section 12 covering how AI frameworks such as PyTorch and JAX expose and represent these types within their own tensor systems, Section 13 showing how the compiler stack underneath those frameworks — a graph compiler, MLIR, and a general-purpose LLVM compiler — actually lowers that computation onto the hardware, with MLIR carrying most of the low-precision-specific work, and Section 14 showing how standardization (IEEE P3109) is still racing to catch up with formats that are already shipping.

As stated at the outset, none of this is new: every format, every formula, and every hardware

number in this survey is drawn from and cited to an existing standard, specification, or vendor publication. Its only contribution is to have gathered that material into one self-contained, consistently notated document, so that a reader can walk from “what is a floating-point number” to “how does a Blackwell GPU multiply two NVFP4 tensors” without leaving the page. Given how quickly this particular corner of computer arithmetic is still moving — NVFP4 KV-cache compression and FP4 training were both, at the time of writing, recent developments rather than settled practice, and IEEE P3109 remains unratiﬁed — readers should expect the newest sections of this survey in particular to need revisiting as the field continues to move.

References

- [1] ARITH 2025 Proceedings. Formal verification of the IEEE P3109 standard for binary floating-point formats for machine learning. In *Proceedings of the IEEE Symposium on Computer Arithmetic (ARITH)*, 2025. URL <https://arith2025.org/proceedings/215900a157.pdf>.
- [2] Colfax Research. CUTLASS tutorial: Sub-byte GEMM on NVIDIA Blackwell GPUs, 2025. URL <https://research.colfax-intl.com/cutlass-tutorial-sub-byte-gemm-on-nvidia-blackwell-gpus/>. Accessed September 2026.
- [3] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference. *arXiv preprint arXiv:2103.13630*, 2021.
- [4] David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.
- [5] IEEE Computer Society. IEEE standard for binary floating-point arithmetic. Technical Report ANSI/IEEE Std 754-1985, Institute of Electrical and Electronics Engineers, 1985.
- [6] IEEE Computer Society. IEEE standard for radix-independent floating-point arithmetic. Technical Report IEEE Std 854-1987, Institute of Electrical and Electronics Engineers, 1987.
- [7] IEEE Computer Society. IEEE standard for floating-point arithmetic. Technical Report IEEE Std 754-2019 (Revision of IEEE 754-2008), Institute of Electrical and Electronics Engineers, 2019.
- [8] IEEE P3109 Working Group. P3109 – arithmetic formats for machine learning, 2026. URL <https://sagroups.ieee.org/p3109wgpublic/>.
- [9] JAX Authors. ml_dtypes: A stand-alone implementation of several NumPy dtype extensions used in machine learning, 2026. URL https://github.com/jax-ml/ml_dtypes/blob/main/README.md. Accessed September 2026.
- [10] LLVM Discourse. Random number generation in MLIR?, 2020. URL <https://discourse.llvm.org/t/random-number-generation-in-mlir/2381>. Posted December 2020.
- [11] LLVM Project. [NVPTX] add conversion intrinsics from/to fp8 types (e4m3, e5m2), 2024. URL <https://www.mail-archive.com/cfe-commits@lists.llvm.org/msg465721.html>. Posted August 2024.
- [12] LLVM Project. [MLIR][NVVM] add RN/RZ rounding modes to f16x2/bf16x2 conversions, 2025. URL <https://github.com/llvm/llvm-project/pull/169005>. Merged November 27, 2025.
- [13] LLVM Project. llvm/lib/support/APFloat.cpp, 2026. URL <https://github.com/llvm/llvm-project/blob/main/llvm/lib/Support/APFloat.cpp>. Accessed September 2026; see the `semFloat8E4M3FN`, `semFloat6E2M3FN`, `semFloat4E2M1FN`, and related semantics definitions.
- [14] LLVM Project. mlir/lib/dialect/MemRef/transforms/EmulateNarrowType.cpp, 2026. URL <https://codebrowser.dev/llvm/mlir/lib/Dialect/MemRef/Transforms/EmulateNarrowType.cpp.html>. Accessed September 2026.
- [15] LLVM Project. llvm/include/llvm/IR/Type.h, 2026. URL https://llvm.org/doxygen/IR_2Type_8h_source.html. Accessed September 2026; see the `Type::TypeID` enumeration.
- [16] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David

- Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. *arXiv preprint arXiv:1710.03740*, 2018.
- [17] Paulius Micikevicius, Dusan Stolic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, Naveen Mellempudi, Stuart Oberman, Mohammad Shoeybi, Michael Siu, and Hao Wu. FP8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.
 - [18] Microsoft Corporation. microxcaling: A library for mx (microscaling) quantization, 2023. URL <https://github.com/microsoft/microxcaling>.
 - [19] MLIR. Builtin dialect – MLIR, 2026. URL <https://mlir.llvm.org/docs/Dialects/Builtin/>. Accessed September 2026; see the Float8E4M3Type, Float6E2M3FNType, Float4E2M1FNType, and related builtin floating-point types.
 - [20] MLIR. Dialects – MLIR, 2026. URL <https://mlir.llvm.org/docs/Dialects/>. Accessed September 2026; full list of upstream MLIR dialects.
 - [21] MLIR. ‘quant’ dialect – MLIR, 2026. URL <https://mlir.llvm.org/docs/Dialects/QuantDialect/>. Accessed September 2026.
 - [22] NVIDIA, Felix Abecassis, Anjulie Agrusa, Dong Ahn, Jonah Alben, Mike Chrzanowski, Eric Chung, Steve Dai, Bitu Darvish Rouhani, Carlo del Mundo, Brucek Khailany, Paulius Micikevicius, et al. Pretraining large language models with NVFP4. *arXiv preprint arXiv:2509.25149*, 2025.
 - [23] NVIDIA Corporation. TensorFloat-32 in the A100 GPU accelerates AI training, HPC up to 20x, 2020. URL <https://developer.nvidia.com/blog/tensorfloat-32-precision-format/>.
 - [24] NVIDIA Corporation. Introducing NVFP4 for efficient and accurate low-precision inference, 2025. URL <https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/>.
 - [25] NVIDIA Corporation. Blackwell SM100 GEMMs — CUTLASS documentation, 2026. URL https://docs.nvidia.com/cutlass/4.3.1/media/docs/cpp/blackwell_functionality.html. Accessed September 2026.
 - [26] NVIDIA Corporation. Native FP8 — JAX toolbox documentation, 2026. URL https://github.com/NVIDIA/JAX-Toolbox/blob/main/docs/NATIVE_FP8.md. Accessed September 2026.
 - [27] NVIDIA Corporation. Parallel thread execution ISA (PTX ISA), 2026. URL <https://docs.nvidia.com/cuda/parallel-thread-execution/>. Accessed September 2026.
 - [28] NVIDIA Corporation. MXFP8 — transformer engine documentation, 2026. URL https://nvidia.github.io/TransformerEngine/features/low_precision_training/mx_fp8/mx_fp8.html. Accessed September 2026.
 - [29] NVIDIA Corporation. NVFP4 — transformer engine documentation, 2026. URL https://nvidia.github.io/TransformerEngine/features/low_precision_training/nvfp4/nvfp4.html. Accessed September 2026.
 - [30] NVIDIA Research. Pushing intelligence to 4-bit, 2025. URL <https://research.nvidia.com/labs/eai/blogs/pushing-intelligence-to-4-bit/>.
 - [31] Open Compute Project. OCP microscaling formats (MX) specification v1.0, 2023. URL <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf>.

- [32] Open Compute Project. AMD, Arm, Intel, Meta, Microsoft, NVIDIA, and Qualcomm standardize next-generation narrow precision data formats for AI, 2023. URL <https://www.opencompute.org/blog/amd-arm-intel-meta-microsoft-nvidia-and-qualcomm-standardize-next-generation-narrow-precision-data-formats-for-ai>.
- [33] OpenXLA. [RFC] FP8 in XLA, 2022. URL <https://github.com/openxla/xla/discussions/22>. Posted November 2022.
- [34] Nikhil Patel, Michael Lazos, Driss Guessous, and Elias Ellison. Generating state-of-the-art GEMMs with TorchInductor’s CuteDSL backend, 2026. URL <https://pytorch.org/blog/gemms-torchinductor-cutedsl-backend/>. Published April 2026.
- [35] PyTorch. RFC-0030: Proposal of fp8 dtype introduction to PyTorch, 2023. URL <https://github.com/pytorch/rfcs/blob/master/RFC-0030-native-fp8-dtype.md>. Accessed September 2026.
- [36] PyTorch. Add torch.float4_e2m1fn_x2 to PyTorch (pull request #146578), 2025. URL <https://github.com/pytorch/pytorch/pull/146578>. Accessed September 2026.
- [37] PyTorch. MX basic dtypes in pytorch/pytorch (issue #146414), 2025. URL <https://github.com/pytorch/pytorch/issues/146414>. Accessed September 2026.
- [38] PyTorch. torchao: NVFP4Tensor source (nvfp4_tensor.py), 2026. URL https://github.com/pytorch/ao/blob/main/torchao/prototype/mx_formats/nvfp4_tensor.py. Accessed September 2026.
- [39] PyTorch. Tensor attributes — PyTorch documentation, 2026. URL https://docs.pytorch.org/docs/stable/tensor_attributes.html. Accessed September 2026.
- [40] PyTorch Developer Community. Float8 in PyTorch [1/x], 2023. URL <https://dev-discuss.pytorch.org/t/float8-in-pytorch-1-x/1815>. Accessed September 2026.
- [41] ROCm. [Triton/Gluon] [Kernel] add gfx950 stochastic MXFP4 quantization, 2026. URL <https://github.com/ROCm/aiter/pull/5531>. Accessed September 2026.
- [42] Bitu Darvish Rouhani, Ritchie Zhao, Venmugil Elango, Rasoul Shafipour, Mathew Hall, Maral Mesmakhosroshahi, Ankit More, Levi Melnick, Maximilian Golub, Girish Varatkar, et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.
- [43] Semiconductor Engineering. Will floating point 8 solve AI/ML overhead?, 2024. URL <https://semiengineering.com/will-floating-point-8-solve-ai-ml-overhead/>.
- [44] Triton. Block-scaled matrix multiplication, 2026. URL <https://triton-lang.org/main/getting-started/tutorials/gluon/tcgen05-mma-scaled.html>. Accessed September 2026.
- [45] Shibo Wang and Pankaj Kanwar. BFloat16: The secret to high performance on cloud tpus, 2019. URL <https://cloud.google.com/blog/products/ai-machine-learning/bfloat16-the-secret-to-high-performance-on-cloud-tpus>.